

ГЛАВА 12

ДИНАМИЧЕСКАЯ ПАМЯТЬ

В этой главе...

12.1. Динамическая память и интеллектуальные указатели	574
12.2. Динамические массивы	606
12.3. Использование библиотеки: программа запроса текста	617
Резюме	625
Термины	625

Написанные до сих пор программы использовали объекты, имевшие четко определенную продолжительность существования. Глобальные объекты создаются при запуске программы и освобождаются по завершении выполнения программы. Локальные автоматические объекты создаются при входе в блок, где они определены, и удаляются при выходе из него. Статические локальные объекты создаются перед их первым использованием и удаляются по завершении программы.

В дополнение к автоматическим и статическим объектам язык C++ позволяет создавать объекты динамически. Продолжительность существования объектов, созданных динамически, не зависит от того, где они созданы; они существуют, пока не будут освобождены явно.

Процесс освобождения динамических объектов оказывается удивительно богатым источником ошибок. Чтобы сделать использование динамических объектов безопасней, библиотека определяет два типа интеллектуальных указателей, управляющих динамическим созданием объектов. Интеллектуальные указатели гарантируют, что объекты, на которые они указывают, будут автоматически освобождены в соответствующий момент.

До сих пор наши программы использовали только статические объекты или объекты, располагаемые в стеке. Статическая память используется для локальных статических переменных (см. раздел 6.1.1, стр. 272), для статических переменных-членов классов (см. раздел 7.6, стр. 387), а также для переменных, определенных вне функций. Стек используется для нестатических объектов, определенных в функциях. Объекты, расположенные в статической памяти или в стеке, автоматически создаются и удаляются компилятором. Объекты из

стека существуют, только пока выполняется блок, в котором они определены; статические объекты создаются прежде, чем они будут использованы, и удаляются по завершении программы.

Кроме статической памяти и стека, у каждой программы есть также пул памяти, которую она может использовать. Это *динамическая память* (free store) или *распределяемая память* (heap). Программы используют распределяемую память для объектов, называемых *динамически созданными объектами* (dynamically allocated object), место для которых программа резервирует во время выполнения. Программа сама контролирует продолжительность существования динамических объектов; наш код должен явно освобождать такие объекты, когда они больше не нужны.



Хотя динамическая память иногда необходима, ее корректное освобождение зачастую довольно сложно.

12.1. Динамическая память и интеллектуальные указатели

Для управления динамической памятью в языке C++ используются два оператора: *оператор new*, который резервирует (а при необходимости и инициализирует) объект в динамической памяти и возвращает указатель на него; и *оператор delete*, который получает указатель на динамический объект и удаляет его, освобождая зарезервированную память.

Работа с динамической памятью проблематична, поскольку на удивление сложно гарантировать освобождение памяти в нужный момент. Если забыть освобождать память, то появится утечка памяти, если освободить область памяти слишком рано, пока еще есть указатели на нее, то получится указатель на не существующую более область памяти.



Чтобы сделать использование динамической памяти проще (и безопасный), новая библиотека предоставляет два типа *интеллектуальных указателей* (smart pointer) для управления динамическими объектами. Интеллектуальный указатель действует, как обычный указатель, но с важным дополнением: автоматически удаляет объект, на который он указывает. Новая библиотека определяет два вида интеллектуальных указателей, отличающихся способом управления своими базовыми указателями: указатель `shared_ptr` позволяет нескольким указателям указывать на тот же объект, а указатель `unique_ptr` — нет. Библиотека определяет также сопутствующий класс `weak_ptr`, являющийся второстепенной ссылкой на объект, управляемый указателем `shared_ptr`. Все три класса определены в заголовке `memory`.

12.1.1. Класс `shared_ptr`

**C++
11**

Подобно векторам, интеллектуальные указатели являются шаблонами (см. раздел 3.3, стр. 141). Поэтому при создании интеллектуального указателя следует предоставить дополнительную информацию — в данном случае тип, на который способен указывать указатель. Подобно векторам, этот тип указывают в угловых скобках, следующих за именем типа определяемого интеллектуального указателя:

```
shared_ptr<string> p1;    // shared_ptr может указывать на строку
shared_ptr<list<int>> p2; // shared_ptr может указывать на
                        // список целых чисел
```

Инициализированный по умолчанию интеллектуальный указатель хранит нулевой указатель (см. раздел 2.3.2, стр. 89). Дополнительные способы инициализации интеллектуального указателя рассматриваются в разделе 12.1.3 (стр. 591).

Интеллектуальный указатель используется теми же способами, что и обычный указатель. Обращение к значению интеллектуального указателя возвращает объект, на который он указывает. Когда интеллектуальный указатель используется в условии, результат проверки может засвидетельствовать, не является ли он нулевым:

```
// если указатель p1 не нулевой и не указывает на пустую строку
if (p1 && p1->empty())
    *p1 = "hi"; // обратиться к значению p1, чтобы присвоить ему
                // новое значение строки
```

Список общих функций указателей `shared_ptr` и `unique_ptr` приведен в табл. 12.1. Функции, специфические для указателя `shared_ptr`, перечислены в табл. 12.2.

Таблица 12.1. Функции, общие для указателей `shared_ptr` и `unique_ptr`

<code>shared_ptr<T> sp</code> <code>unique_ptr<T> up</code>	Нулевой интеллектуальный указатель, способный указывать на объекты типа <code>T</code>
<code>p</code>	При использовании указателя <code>p</code> в условии возвращается значение <code>true</code> , если он указывает на объект
<code>*p</code>	Обращение к значению указателя <code>p</code> возвращает объект, на который он указывает
<code>p->mem</code>	Синоним для <code>(*p).mem</code>
<code>p.get()</code>	Возвращает указатель, хранимый указателем <code>p</code> . Используйте его осторожно, поскольку объект, на который он указывает, может прекратить существование после удаления его интеллектуальным указателем
<code>swap(p, q)</code> <code>p.swap(q)</code>	Обменивает указатели в <code>p</code> и <code>q</code>

Таблица 12.2. Функции, специфические для указателя `shared_ptr`

<code>make_shared<T>(args)</code>	Возвращает указатель <code>shared_ptr</code> на динамически созданный объект типа <code>T</code> . Аргументы <code>args</code> используются для инициализации создаваемого объекта
<code>shared_ptr<T> p(q)</code>	<code>p</code> — копия <code>shared_ptr q</code> ; инкремент счетчика <code>q</code> . Тип содержащегося в <code>q</code> указателя должен быть приводим к типу <code>T*</code> (см. раздел 4.11.2, стр. 221)
<code>p = q</code>	<code>p</code> и <code>q</code> — указатели <code>shared_ptr</code> , содержащие указатели, допускающие приведение друг к другу. Происходит декремент счетчика ссылок <code>p</code> и инкремент счетчика <code>q</code> ; если счетчик указателя <code>p</code> достиг 0, память его объекта освобождается
<code>p.unique()</code>	Возвращает <code>true</code> , если <code>p.use_count()</code> равно единице, и значение <code>false</code> в противном случае
<code>p.use_count()</code>	Возвращает количество объектов, совместно использующих указатель <code>p</code> ; может выполняться очень медленно, предназначена прежде всего для отладки

Функция `make_shared()`

Наиболее безопасный способ резервирования и использования динамической памяти подразумевает вызов библиотечной функции `make_shared()`. Она резервирует и инициализирует объект в динамической памяти, возвращая указатель типа `shared_ptr` на этот объект. Как и типы интеллектуальных указателей, функция `make_shared()` определена в заголовке `memory`.

При вызове функции `make_shared()` следует указать тип создаваемого объекта. Это подобно использованию шаблона класса — за именем функции следует указание типа в угловых скобках:

```
// указатель shared_ptr на объект типа int со значением 42
shared_ptr<int> p3 = make_shared<int>(42);
// p4 указывает на строку со значением '9999999999'
shared_ptr<string> p4 = make_shared<string>(10, '9');
// p5 указывает на объект типа int со значением по
// умолчанию (р 3.3.1, стр. 142) 0
shared_ptr<int> p5 = make_shared<int>();
```

Подобно функции-члену `emplace()` последовательного контейнера (см. раздел 9.3.1, стр. 444), функция `make_shared()` использует свои аргументы для создания объекта заданного типа. Например, при вызове функции `make_shared<string>()` следует передать аргумент (аргументы), соответствующий одному из конструкторов типа `string`. Вызову функции `make_shared<int>()` можно передать любое значение, которое можно использовать для инициализации переменной типа `int`, и т.д. Если не передать аргументы, то объект инициализируется значением по умолчанию (см. раздел 3.3.1, стр. 143).

Для облегчения определения объекта, содержащего результат вызова функции `make_shared()`, обычно используют ключевое слово `auto` (см. раздел 2.5.2, стр. 107):

```
// p6 указывает на динамически созданный пустой вектор vector<string>
auto p6 = make_shared<vector<string>>();
```

Копирование и присвоение указателей `shared_ptr`

При копировании и присвоении указателей `shared_ptr` каждый из них отслеживает количество других указателей `shared_ptr` на тот же объект:

```
auto p = make_shared<int>(42); // объект, на который указывает p
                                // имеет только одного владельца
auto q(p); // p и q указывают на тот же объект
// объект, на который указывают p и q, имеет двух владельцев
```

С указателем `shared_ptr` связан счетчик, обычно называемый *счетчиком ссылок* (reference count). При копировании указателя `shared_ptr` значение счетчика увеличивается. Например, значение связанного с указателем `shared_ptr` счетчика увеличивается, когда он используется для инициализации другого указателя `shared_ptr`, а также при использовании его в качестве правого операнда присвоения, или при передаче его функции (см. раздел 6.2.1, стр. 277), или при возвращении из функции по значению (см. раздел 6.3.2, стр. 294). Значение счетчика увеличивается при присвоении нового значения указателю `shared_ptr`, а когда он удаляется или когда локальный указатель `shared_ptr` выходит из области видимости (см. раздел 6.1.1, стр. 271), значение счетчика уменьшается.

Как только счетчик указателя `shared_ptr` достигает нуля, он автоматически освобождает объект, на который указывает:

```
auto r = make_shared<int>(42); // объект int, на который указывает r,
                                // имеет одного владельца
r = q; // присвоение r переводит этот указатель на другой адрес
        // приращение счетчика владельцев объекта, на который указывает q
        // уменьшение счетчика владельцев объекта, на который указывает r
        // объект, на который указывал r, не имеет более владельцев;
        // он освобождается автоматически
```

Здесь резервируется переменная типа `int`, а ее адрес сохраняется в указателе `r`. Затем указателю `r` присваивается новое значение. В данном случае `r` — единственный указатель типа `shared_ptr`, указывающий на этот объект. В результате присвоения `r = q` переменная `int` автоматически освобождается.



Будет ли использован счетчик или другая структура данных для отслеживания количества указателей на совместно используемый объект, зависит от реализации компилятора. Главное то, что класс отслеживает количество указателей `shared_ptr` на тот же объект и автоматически освобождает его в подходящий момент.

Указатель `shared_ptr` автоматически удаляет свои объекты...

Когда последний указатель `shared_ptr` на объект удаляется, его класс автоматически удаляет объект, на который он указывает. Для этого используется другая специальная функция-член — *деструктор* (destructor), аналогичная конструкторам, которые есть у каждого класса. Подобно тому, как конструктор контролирует инициализацию, деструктор контролирует происходящее при удалении объектов этого типа.

Деструкторы обычно освобождают ресурсы, зарезервированные объектом. Например, конструкторы класса `string` (как и другие его члены) резервируют память для содержания составляющих ее символов. Деструктор класса `string` освобождает эту память. Точно так же некоторые функции класса `vector` резервируют память для хранения элементов вектора. Деструктор класса `vector` удаляет эти элементы и освобождает используемую ими память.

Деструктор указателя `shared_ptr` осуществляет декремент счетчика ссылок объекта, на который он указывает. Когда счетчик достигает нуля, деструктор указателя `shared_ptr` удаляет объект, на который он указывает, и освобождает используемую им память.

...и автоматически освобождает их память

Тот факт, что класс `shared_ptr` автоматически освобождает динамические объекты, когда они больше не нужны, существенно облегчает использование динамической памяти. Рассмотрим, например, функцию, которая возвращает указатель `shared_ptr` на динамически созданный объект типа `Foo`, который может быть инициализирован аргументом типа `T`:

```
// функция factory() возвращает указатель shared_ptr на динамически
// созданный объект
shared_ptr<Foo> factory(T arg)
{
    // обработать аргумент соответствующим образом
    // shared_ptr позаботится об освобождении этой памяти
    return make_shared<Foo>(arg);
}
```

Функция `factory()` возвращает указатель `shared_ptr`, гарантирующий удаление созданного ею объекта в подходящий момент. Например, следующая функция сохраняет указатель `shared_ptr`, возвращенный функцией `factory()`, в локальной переменной:

```
void use_factory(T arg)
{
    shared_ptr<Foo> p = factory(arg);
    // использует p
} // p выходит из области видимости; память, на которую он указывал,
// освобождается автоматически
```

Поскольку указатель `p` является локальным для функции `use_factory()`, он удаляется по ее завершении (см. раздел 6.1.1, стр. 271). Когда указатель `p` удаляется, осуществляется декремент его счетчика ссылок и проверка. В данном случае `p` — единственный указатель на объект в памяти, возвращенный функцией `factory()`. Поскольку указатель `p` выходит из области видимости, объект, на который он указывает, удаляется, а память, в которой он располагался, освобождается.

Память не будет освобождена, если на нее будет указывать любой другой указатель типа `shared_ptr`:

```
shared_ptr<Foo> use_factory(T arg)
{
    shared_ptr<Foo> p = factory(arg);
    // использует p
    return p; // при возвращении p счетчик ссылок увеличивается
} // p выходит из области видимости; память, на которую он указывал,
// не освобождается
```

В этой версии функции `use_factory()` оператор `return` возвращает вызывающей стороне (см. раздел 6.3.2, стр. 294) копию указателя `p`. Копирование указателя `shared_ptr` добавляет единицу к счетчику ссылок этого объекта. Теперь, когда указатель `p` удаляется, останется другой владелец области памяти, на которую указывал указатель `p`. Класс `shared_ptr` гарантирует, что пока есть хоть один указатель `shared_ptr` на данную область памяти, она не будет освобождена.

Поскольку память не освобождается, пока не удалится последний указатель `shared_ptr`, важно гарантировать, что ни одного указателя `shared_ptr` не останется после того, как необходимость в них отпадет. Если не удалить ненужный указатель `shared_ptr`, программа будет выполняться правильно, но может впустую тратить память. Одна из возможностей оставить указатели `shared_ptr` после употребления — поместить их в контейнер, а затем переупорядочить его так, чтобы эти элементы оказались не нужны. Поэтому важно гарантировать удаление элементов с указателями `shared_ptr`, как только они больше не нужны.



Если указатели `shared_ptr` помещаются в контейнер, но впоследствии будут использованы лишь некоторые из них, а не все, то следует не забыть самостоятельно удалить остальные элементы.

Классы, ресурсы которых имеют динамическую продолжительность существования

Обычно динамическую память используют в следующих случаях.

1. Неизвестно необходимое количество объектов.
2. Неизвестен точный тип необходимых объектов.
3. Нельзя разрешать совместное использование данных несколькими объектами.

Классы контейнеров — хороший пример классов, использующих динамическую память, как в первом случае. Примеры второго рассматриваются в главе 15. В данном разделе определяется класс, использующий динамическую память для того, чтобы позволить нескольким объектам совместно использовать те же данные.

Использованные до сих пор классы резервировали ресурсы, которые существовали, только пока существовал объект. Например, каждому вектору принадлежат его собственные элементы. При копировании вектора элементы исходного вектора копировались в независимые элементы другого:

```
vector<string> v1; // пустой вектор
{ // новая область видимости
    vector<string> v2 = {"a", "an", "the"};
    v1 = v2; // копирует элементы из v2 в v1
} // v2 удаляется, что удаляет элементы v2
// v1 содержит три элемента, являющихся копиями элементов v2
```

Элементы вектора существуют, только пока существует сам вектор. Когда вектор удаляется, удаляются и его элементы.

Некоторые классы резервируют ресурсы, продолжительность существования которых не зависит от первоначального объекта. Например, необходимо определить класс Blob, содержащий коллекцию элементов. В отличие от контейнеров, объекты класса Blob должны быть копиями друг друга и совместно использовать те же элементы. Таким образом, при копировании объекта класса Blob элементы копии должны ссылаться на те же элементы, что и оригинал.

Обычно, когда два объекта совместно используют те же данные, они не удаляются при удалении одного из объектов:

```
Blob<string> b1; // пустой Blob
{ // новая область видимости
    Blob<string> b2 = {"a", "an", "the"};
    b1 = b2; // b1 и b2 совместно используют те же элементы
} // b2 удаляется, но элементы b2 нет
// b1 указывает на элементы, первоначально созданные в b2
```

В этом примере объекты b1 и b2 совместно используют те же элементы. Когда объект b2 выходит из области видимости, эти элементы должны остаться, поскольку объект b1 все еще использует их.

Основная причина использования динамической памяти в том, чтобы позволить нескольким объектам совместно использовать те же данные.

Определение класса StrBlob

В конечном счете класс Blob будет реализован как шаблон, но это только в разделе 16.1.2 (стр. 830), а пока определим его версию, способную манипулировать только строками. Поэтому назовем данную версию этого класса StrBlob.

Простейший способ реализации нового типа коллекции подразумевает использование одного из библиотечных контейнеров. Это позволит библиотечному типу управлять собственно хранением элементов. В данном случае для хранения элементов будет использован класс `vector`.

Однако сам вектор не может храниться непосредственно в объекте `Blob`. Члены объекта удаляются при удалении самого объекта. Предположим, например, что объекты `b1` и `b2` класса `Blob` совместно используют тот же вектор. Если бы вектор хранился в одном из этих объектов, скажем в `b2`, то, как только объект `b2` выйдет из области видимости, элементы вектора перестанут существовать. Чтобы гарантировать продолжение существования элементов, будем хранить вектор в динамической памяти.

Для реализации совместного использования снабдим каждый объект класса `StrBlob` указателем `shared_ptr` на вектор в динамической памяти. Указатель-член `shared_ptr` будет следить за количеством объектов класса `StrBlob`, совместно использующих тот же самый вектор, и удалит его, когда будет удален последний объект класса `StrBlob`.

Осталось решить, какие функции будет предоставлять создаваемый класс. Реализуем пока небольшое подмножество функций вектора. Изменим также функции обращения к элементам (включая `front()` и `back()`): в данном классе при попытке доступа к не существующим элементам они будут передавать исключения.

У класса будет стандартный конструктор и конструктор с параметром типа `initializer_list<string>` (см. раздел 6.2.6, стр. 290). Этот конструктор будет получать список инициализаторов в скобках.

```
class StrBlob {
public:
    typedef std::vector<std::string>::size_type size_type;
    StrBlob();
    StrBlob(std::initializer_list<std::string> il);
    size_type size() const { return data->size(); }
    bool empty() const { return data->empty(); }
    // добавление и удаление элементов
    void push_back(const std::string &t) {data->push_back(t);}
    void pop_back();
    // доступ к элементам
    std::string& front();
    std::string& back();
private:
    std::shared_ptr<std::vector<std::string>> data;
    // передать сообщение при недопустимости data[i]
    void check(size_type i, const std::string &msg) const;
};
```

В классе будут реализованы функции-члены `size()`, `empty()` и `push_back()`, которые передают свою работу через указатель `data` внутреннему вектору. Например, функция `size()` класса `StrBlob` вызывает функцию `data->size()` и т.д.

Конструкторы класса **StrBlob**

Для инициализации своей переменной-члена `data` указателем на динамически созданный вектор каждый конструктор использует собственный список инициализации (см. раздел 7.1.4, стр. 345). Стандартный конструктор резервирует пустой вектор:

```
StrBlob::StrBlob(): data(make_shared<vector<string>>()) { }
StrBlob::StrBlob(initializer_list<string> il):
    data(make_shared<vector<string>>(il)) { }
```

Конструктор, получающий тип `initializer_list`, передает свой параметр для соответствующего конструктора класса `vector` (см. раздел 2.2.1, стр. 77). Этот конструктор инициализирует элементы вектора копиями значений из списка.

Функции-члены доступа к элементам

Функции `pop_back()`, `front()` и `back()` обращаются к соответствующим функциям-членам вектора. Эти функции должны проверять существование элементов прежде, чем попытаться получить доступ к ним. Поскольку несколько функций-членов должны осуществлять ту же проверку, снабдим класс закрытой вспомогательной функцией `check()`, проверяющей принадлежность заданного индекса диапазону. Кроме индекса, функция `check()` получает аргумент типа `string`, передаваемый обработчику исключений. Строка описывает то, что пошло не так, как надо:

```
void StrBlob::check(size_type i, const string &msg) const
{
    if (i >= data->size())
        throw out_of_range(msg);
}
```

Функция `pop_back()` и функции-члены доступа к элементам сначала вызывают функцию `check()`. Если проверка успешна, эти функции-члены передают свою работу соответствующим функциям вектора:

```
string& StrBlob::front()
{
    // если вектор пуст, функция check() передаст следующее
    check(0, "front on empty StrBlob");
    return data->front();
}

string& StrBlob::back()
{
    check(0, "back on empty StrBlob");
    return data->back();
}

void StrBlob::pop_back()
```

```
{  
    check(0, "pop_back on empty StrBlob");  
    data->pop_back();  
}
```

Функции-члены `front()` и `back()` должны быть перегружены для констант (см. раздел 7.3.2, стр. 359). Определение этих версий остается в качестве самостоятельного упражнения.

Копирование, присвоение и удаление объектов класса `StrBlobs`

Подобно классу `Sales_data`, класс `StrBlob` использует стандартные версии функций копирования, присвоения и удаления объектов (см. раздел 7.1.5, стр. 347). По умолчанию эти функции копируют, присваивают и удаляют переменные-члены класса. У класса `StrBlob` есть только одна переменная-член — указатель `shared_ptr`. Поэтому при копировании, присвоении и удалении объекта класса `StrBlob` его переменная-член `shared_ptr` будет скопирована, присвоена или удалена.

Как уже упоминалось выше, копирование указателя `shared_ptr` приводит к инкременту его счетчика ссылок; присвоение одного указателя `shared_ptr` другому приводит к инкременту счетчика правого операнда и декременту счетчика левого; удаление указателя `shared_ptr` приводит к декременту его счетчика. Если значение счетчика указателя `shared_ptr` доходит до нуля, объект, на который он указывает, удаляется автоматически. Таким образом, вектор, созданный конструкторами класса `StrBlob`, будет автоматически удален при удалении последнего объекта класса `StrBlob`, указывающего на этот вектор.

Упражнения раздела 12.1.1

Упражнение 12.1. Сколько элементов будут иметь объекты `b1` и `b2` в конце этого кода?

```
StrBlob b1;  
{  
    StrBlob b2 = {"a", "an", "the"};  
    b1 = b2;  
    b2.push_back("about");  
}
```

Упражнение 12.2. Напишите собственную версию класса `StrBlob`, включающего константные версии функций `front()` и `back()`.

Упражнение 12.3. Нуждается ли этот класс в константных версиях функций `push_back()` и `pop_back()`? Если они нужны, добавьте их. В противном случае объясните, почему они не нужны?

Упражнение 12.4. В функции `check()` нет проверки того, что параметр `i` больше нуля. Почему эта проверка не нужна?

Упражнение 12.5. Конструктор, получающий тип `initializer_list`, не был объявлен как `explicit` (см. раздел 7.5.4, стр. 383). Обсудите преимущества и недостатки этого выбора.

12.1.2. Непосредственное управление памятью

Язык определяет два оператора, позволяющие резервировать и освободить области в динамической памяти. Оператор `new` резервирует память, а оператор `delete` освобождает память, зарезервированную оператором `new`.

По причинам, которые станут ясны позже, использование этих операторов для управления памятью существенно более подвержено ошибкам, чем использование интеллектуальных указателей. Кроме того, классы, самостоятельно управляющие памятью (в отличие от таковых, использующих интеллектуальные указатели), не могут полагаться на стандартные определения тех их членов, которые копируют, присваивают и удаляют объекты класса (см. раздел 7.1.4, стр. 345). В результате программы, использующие интеллектуальные указатели, вероятно, будет проще написать и отлаживать.



ВНИМАНИЕ

Пока не пройдена глава 13, будем использовать классы, резервирующие динамическую память, *только если* для управления ею используются интеллектуальные указатели.

Использование оператора `new` для динамического резервирования и инициализации объектов

Созданные в динамической памяти объекты не имеют имен, поэтому оператор `new` не предполагает никаких способов именования резервируемых объектов. Вместо этого оператор `new` возвращает указатель на зарезервированный объект:

```
int *pi = new int; // pi указывает на динамически созданный, безымянный,
                  // неинициализированный объект типа int
```

Это выражение `new` создает в динамической памяти объект типа `int` и возвращает указатель на него.

По умолчанию создаваемые в динамической памяти объекты инициализируются значением по умолчанию (см. раздел 2.2.1 стр. 77). Это значит, что у объектов встроенного или составного типа будет неопределенное значение, а объекты типа класса инициализируются их стандартным конструктором:

```
string *ps = new string; // инициализируется пустой строкой
int *pi = new int;       // pi указывает на неинициализированный int
```


Тип `p1` — это указатель на автоматически выведенный тип `obj`. Если `obj` имеет тип `int`, то тип `p1` — `int*`; если `obj` имеет тип `string`, то тип `p1` — `string*` и т.д. Вновь созданный объект инициализируется значением объекта `obj`.

Динамически созданные константные объекты

Для резервирования константных объектов вполне допустимо использовать оператор `new`:

```
// зарезервировать и инициализировать const int
const int *pci = new const int(1024);
// зарезервировать и инициализировать значением по умолчанию const string
const string *pcs = new const string;
```

Подобно любым другим константным объектам, динамически созданный константный объект следует инициализировать. Динамический константный объект типа класса, определяющего стандартный конструктор (см. раздел 7.1.4, стр. 343), можно инициализировать неявно. Объекты других типов следует инициализировать явно. Поскольку динамически зарезервированный объект является константой, возвращенный оператором `new` указатель является указателем на константу (см. раздел 2.4.2, стр. 99).

Исчерпание памяти

Хотя современные машины имеют огромный объем памяти, всегда существует вероятность исчерпания динамической памяти. Как только программа использует всю доступную ей память, выражения с оператором `new` будут терпеть неудачу. По умолчанию, если оператор `new` неспособен зарезервировать требуемый объем памяти, он передает исключение типа `bad_alloc` (см. раздел 5.6, стр. 257). Используя иную форму оператора `new`, можно воспрепятствовать передаче исключения:

```
// при неудаче оператор new возвращает нулевой указатель
int *p1 = new int; // при неудаче оператор new передает
                  // исключение std::bad_alloc
int *p2 = new (nothrow) int; // при неудаче оператор new возвращает
                             // нулевой указатель
```

По причинам, рассматриваемым в разделе 19.1.2 (стр. 1028), эта форма оператора `new` упоминается как *размещающий оператор* `new` (`placement new`). Выражение размещающего оператора `new` позволяет передать дополнительные аргументы. В данном случае передается определенный библиотекой объект `nothrow`. Передача объекта `nothrow` оператору `new` указывает, что он не должен передавать исключения. Если эта форма оператора `new` окажется неспособна зарезервировать требуемый объем памяти, она возвратит нулевой указатель. Типы `bad_alloc` и `nothrow` определены в заголовке `new`.

Освобождение динамической памяти

Чтобы предотвратить исчерпание памяти, по завершении использования ее следует вернуть операционной системе. Для этого используется *оператор delete*, получающий указатель на освобождаемый объект:

```
delete p; // p должен быть указателем на динамически созданный объект
         // или нулевым указателем
```

Подобно оператору *new*, оператор *delete* выполняет два действия: удаляет объект, на который указывает переданный ему указатель, и освобождает соответствующую область памяти.

Значения указателя и оператор *delete*

Передаваемый оператору *delete* указатель должен либо указывать на динамически созданный объект, либо быть нулевым указателем (см. раздел 2.3.2, стр. 89). Результат удаления указателя на область памяти, зарезервированную не оператором *new*, или повторного удаления значения того же указателя непредсказуем:

```
int i, *pi1 = &i, *pi2 = nullptr;
double *pd = new double(33), *pd2 = pd;
delete i;    // ошибка: i - не указатель
delete pi1;  // непредсказуемо: pi1 - локальный
delete pd;   // ok
delete pd2;  // непредсказуемо: память, на которую указывает pd2,
              // уже освобождена
delete pi2;  // ok: освобождение нулевого указателя всегда допустимо
```

Компилятор сообщает об ошибке оператора *delete i*, поскольку знает, что *i* — не указатель. Ошибки, связанные с выполнением оператора *delete* для указателей *pi1* и *pd2*, коварней: обычно компиляторы неспособны выяснить, указывает ли указатель на объект, созданный статически или динамически. Точно так же компилятор не может установить, была ли уже освобождена память, на которую указывает указатель. Большинство компиляторов примет такие выражения *delete*, несмотря на их ошибочность.

Хотя значение константного объекта не может быть изменено, сам объект вполне может быть удален. Подобно любым динамическим объектам, константный динамический объект освобождается выполнением оператора *delete* для указателя, указывающего на этот объект:

```
const int *pci = new const int(1024);
delete pci; // ok: удаляет константный объект
```

Динамически созданные объекты существуют до тех пор, пока не будут освобождены

Как упоминалось в разделе 12.1.1 (стр. 578), управляемая указателем `shared_ptr` память автоматически освобождается при удалении последнего указателя `shared_ptr`. Динамический объект, управляемый указателем встроенного типа, существует до тех пор, пока к областям памяти, управляемой при помощи указателей встроенных типов, не будет удален явно.

Функции, возвращающие обычные (а не интеллектуальные) указатели на области динамической памяти, возлагают ответственность за их удаление на вызывающую сторону:

```
// возвращает указатель на динамически созданный объект
Foo* factory(T arg)
{
    // обработать аргумент соответственно
    return new Foo(arg); // за освобождение этой памяти отвечает
                        // вызывающая сторона
}
```

Подобно прежней версии функции `factory()` (см. раздел 12.1.1, стр. 578), эта версия резервирует объект, но не удаляет его. Ответственность за освобождение памяти динамического объекта, когда он станет больше не нужен, несет вызывающая сторона функции `factory()`. К сожалению, вызывающая сторона слишком часто забывает сделать это:

```
void use_factory(T arg)
{
    Foo *p = factory(arg);
    // использовать p, но не удалить его
} // p выходит из области видимости, но память, на которую он указывает,
// не освобождается!
```

Здесь функция `use_factory()` вызывает функцию `factory()` резервирующую новый объект типа `Foo`. Когда функция `use_factory()` завершает работу, локальная переменная `p` удаляется. Эта переменная — встроенный указатель, а не интеллектуальный.

В отличие от классов, при удалении объектов встроенного типа не происходит ничего. В частности, когда указатель выходит из области видимости, с объектом, на который он указывает, ничего не происходит. Если этот указатель указывает на динамическую память, она не освобождается автоматически.



ВНИМАНИЕ

Динамическая память, управляемая при помощи встроенных (а не интеллектуальных) указателей, продолжает существование, пока не будет освобождена явно.

В этом примере указатель `p` был единственным указателем на область памяти, зарезервированную функцией `factory()`. По завершении функции `use_factory()` у программы больше нет никакого способа освободить эту память. Согласно общей логике программирования, следует исправить эту ошибку и напомнить о необходимости освобождения памяти в функции `use_factory()`:

```
void use_factory(T arg)
{
    Foo *p = factory(arg);
    // использование p
    delete p; // не забыть освободить память сейчас, когда
              // она больше не нужна
}
```

Если созданный функцией `use_factory()` объект должен использовать другой код, то эту функцию следует изменить так, чтобы она возвращала указатель на зарезервированную ею память:

```
Foo* use_factory(T arg)
{
    Foo *p = factory(arg);
    // использование p
    return p; // освободить память должна вызывающая сторона
}
```

ВНИМАНИЕ! УПРАВЛЕНИЕ ДИНАМИЧЕСКОЙ ПАМЯТЬЮ ПОДВЕРЖЕНО ОШИБКАМ

Есть три общеизвестных проблемы, связанных с использованием операторов `new` и `delete` при управлении динамической памятью:

1. Память забыли освободить. Когда динамическая память не освобождается, это называется “утечка памяти”, поскольку она уже не возвращается в пул динамической памяти. Проверка утечек памяти очень трудна, поскольку она обычно не проявляется, пока приложение, проработав достаточно долго, фактически не исчерпает память.
2. Объект использован после удаления. Иногда эта ошибка обнаруживается при создании нулевого указателя после удаления.
3. Повторное освобождение той же памяти. Эта ошибка может произойти в случае, когда два указателя указывают на тот же динамически созданный объект. Если оператор `delete` применен к одному из указателей, то память объекта возвращается в пул динамической памяти. Если впоследствии применить оператор `delete` ко второму указателю, то динамическая память может быть нарушена.

Допустить эти ошибки значительно проще, чем потом найти и исправить.



Избежать *всех* этих проблем при использовании исключительно интеллектуальных указателей не получится. Интеллектуальный указатель способен позаботиться об удалении памяти *только* тогда, когда не останется других интеллектуальных указателей на эту область памяти.

Переустановка значения указателя после удаления...

Когда указатель удаляется, он становится недопустимым. Но, даже став недопустимым, на многих машинах он продолжает содержать адрес уже освобожденной области динамической памяти. После освобождения области памяти указатель на нее становится *потерянным указателем* (dangling pointer). Потерянный указатель указывает на ту область памяти, которая когда-то содержала объект, но больше не содержит.

Потерянным указателям присущи все проблемы неинициализированных указателей (см. раздел 2.3.2, стр. 89). Проблем с потерянными указателями можно избежать, освободив связанную с ними память непосредственно перед выходом из области видимости самого указателя. Так не появится шанса использовать указатель уже после того, как связанная с ним память будет освобождена. Если указатель необходимо сохранить, то после применения оператора `delete` ему можно присвоить значение `nullptr`. Это непосредственно свидетельствует о том, что указатель не указывает на объект.

...обеспечивает лишь частичную защиту

Фундаментальная проблема с динамической памятью в том, что может быть несколько указателей на ту же область памяти. Переустановка значения указателя при освобождении памяти позволяет проверять допустимость данного конкретного указателя, но никак не влияет на все остальные указатели, все еще указывающие на уже освобожденную область памяти. Рассмотрим пример:

```
int *p(new int(42)); // p указывает на динамическую память
auto q = p;         // p и q указывают на ту же область памяти
delete p;           // делает недопустимыми p и q
p = nullptr;        // указывает, что указатель p больше не связан с объектом
```

Здесь указатели `p` и `q` указывают на тот же динамически созданный объект. Удалим этот объект и присвоим указателю `p` значение `nullptr`, засвидетельствовав, что он больше не указывает на объект. Однако переустановка значения указателя `p` никак не влияет на указатель `q`, который стал недопустимым после освобождения памяти, на которую указывал указатель `p` (и указатель `q`!). В реальных системах поиск всех указателей на ту же область памяти зачастую на удивление труден.

Упражнения раздела 12.1.2

Упражнение 12.6. Напишите функцию, которая возвращает динамически созданный вектор целых чисел. Передайте этот вектор другой функции, которая читает значения его элементов со стандартного устройства ввода. Передайте вектор другой функции, выводящей прочитанные ранее значения. Не забудьте удалить вектор в подходящий момент.

Упражнение 12.7. Переделайте предыдущее упражнение, используя на сей раз указатель `shared_ptr`.

Упражнение 12.8. Объясните, все ли правильно в следующей функции:

```
bool b() {  
    int* p = new int;  
    // ...  
    return p;  
}
```

Упражнение 12.9. Объясните, что происходит в следующем коде:

```
int *q = new int(42), *r = new int(100);  
r = q;  
auto q2 = make_shared<int>(42), r2 = make_shared<int>(100);  
r2 = q2;
```

12.1.3. Использование указателя `shared_ptr` с оператором `new`

Как уже упоминалось, если не инициализировать интеллектуальный указатель, он инициализируется как нулевой. Как свидетельствует табл. 12.3, интеллектуальный указатель можно также инициализировать указателем, возвращенным оператором `new`:

```
shared_ptr<double> p1; // shared_ptr может указывать на double  
shared_ptr<int> p2(new int(42)); // p2 указывает на int со значением 42
```

Конструкторы интеллектуального указателя, получающие указатели, являются явными (см. раздел 7.5.4, стр. 383). Следовательно, нельзя неявно преобразовать встроенный указатель в интеллектуальный; для инициализации интеллектуального указателя придется использовать прямую форму инициализации (см. раздел 3.2.1, стр. 126):

```
shared_ptr<int> p1 = new int(1024); // ошибка: нужна прямая инициализация  
shared_ptr<int> p2(new int(1024)); // ок: использует прямую инициализацию
```

Таблица 12.3. Другие способы определения и изменения указателя `shared_ptr`

<code>shared_ptr<T> p(q)</code>	Указатель <code>p</code> управляет объектом, на который указывает указатель встроенного типа <code>q</code> ; указатель <code>q</code> должен указывать на область памяти, зарезервированную оператором <code>new</code> , а его тип должен быть преобразуем в тип <code>T*</code>
<code>shared_ptr<T> p(u)</code>	* Указатель <code>p</code> учитывает собственность указателя <code>u</code> типа <code>unique_ptr</code> ; указатель <code>u</code> становится нулевым
<code>shared_ptr<T> p(q, d)</code>	Указатель <code>p</code> учитывает собственность объекта, на который указывает встроенный указатель <code>q</code> . Тип указателя <code>q</code> должен быть преобразуем в тип <code>T*</code> (см. раздел 4.11.2, стр. 221). Для освобождения <code>q</code> указатель <code>p</code> будет использовать вызываемый объект <code>d</code> (см. раздел 10.3.2, стр. 497) вместо оператора <code>delete</code>
<code>shared_ptr<T> p(p2, d)</code>	Указатель <code>p</code> — это копия указателя <code>p2</code> типа <code>shared_ptr</code> , как описано в табл. 12.2, за исключением того, что указатель <code>p</code> использует вызываемый объект <code>d</code> вместо оператора <code>delete</code>
<code>p.reset()</code> <code>p.reset(q)</code> <code>p.reset(q, d)</code>	Если <code>p</code> единственный указатель <code>shared_ptr</code> на объект, функция <code>reset()</code> освободит существующий объект <code>p</code> . Если передан необязательный встроенный указатель <code>q</code> , то <code>p</code> будет указывать на <code>q</code> , в противном случае <code>p</code> станет нулевым. Если предоставлен вызываемый объект <code>d</code> , то он будет вызван для освобождения указателя <code>q</code> , в противном случае используется оператор <code>delete</code>

Инициализация указателя `p1` неявно требует, чтобы компилятор создал указатель типа `shared_ptr` из указателя `int*`, возвращенного оператором `new`. Поскольку нельзя неявно преобразовать обычный указатель в интеллектуальный, такая инициализация ошибочна. По той же причине функция, возвращающая указатель `shared_ptr`, не может неявно преобразовать простой указатель в своем операторе `return`:

```
shared_ptr<int> clone(int p) {
    return new int(p); // ошибка: неявное преобразование
                       // в shared_ptr<int>
}
```

Следует явно связать указатель `shared_ptr` с указателем, который предстоит возвратить:

```
shared_ptr<int> clone(int p) {
    // ok: явное создание shared_ptr<int> из int*
    return shared_ptr<int>(new int(p));
}
```


По умолчанию указатель, используемый для инициализации интеллектуального указателя, должен указывать на область динамической памяти, поскольку по умолчанию интеллектуальные указатели используют оператор `delete` для освобождения связанного с ним объекта. Интеллектуальные указатели можно связать с указателями на другие виды ресурсов. Но для этого необходимо предоставить собственную функцию, используемую вместо оператора `delete`. Предоставление собственного кода удаления рассматривается в разделе 12.1.4 (стр. 597).



Не смешивайте обычные указатели с интеллектуальными

Указатель `shared_ptr` может координировать удаление только с другими указателями `shared_ptr`, которые являются его копиями. Действительно, этот факт — одна из причин, по которой рекомендуется использовать функцию `make_shared()`, а не оператор `new`. Это связывает указатель `shared_ptr` с объектом одновременно с его резервированием. При этом нет никакого способа по неосторожности связать ту же область памяти с несколькими независимо созданными указателями `shared_ptr`.

Рассмотрим следующую функцию, работающую с указателем `shared_ptr`:

```
// ptr создается и инициализируется при вызове process()
void process(shared_ptr<int> ptr)
{
    // использование ptr
} // ptr выходит из области видимости и удаляется
```

Параметр функции `process()` передается по значению, поэтому аргумент копируется в параметр `ptr`. Копирование указателя `shared_ptr` осуществляет инкремент его счетчика ссылок. Таким образом, в функции `process()` значение счетчика не меньше 2. По завершении функции `process()` осуществляется декремент счетчика ссылок указателя `ptr`, но он не может достигнуть нуля. Поэтому, когда локальная переменная `ptr` удаляется, память, на которую она указывает, не освобождается.

Правильный способ использования этой функции подразумевает передачу ей указателя `shared_ptr`:

```
shared_ptr<int> p(new int(42)); // счетчик ссылок = 1
process(p); // копирование p увеличивает счетчик;
           // в функции process() счетчик = 2
int i = *p; // ok: счетчик ссылок = 1
```

Хотя функции `process()` нельзя передать встроенный указатель, ей можно передать временный указатель `shared_ptr`, явно созданный из встроенного указателя. Но это, вероятно, будет ошибкой:

```
int *x(new int(1024)); // опасно: x - обычный указатель, а
                      // не интеллектуальный
process(x); // ошибка: нельзя преобразовать int* в shared_ptr<int>
process(shared_ptr<int>(x)); // допустимо, но память будет освобождена!
int j = *x; // непредсказуемо: x - потерянный указатель!
```

В этом вызове функции `process()` передан временный указатель `shared_ptr`. Этот временный указатель удаляется, когда завершается выражение, в котором присутствует вызов. Удаление временного объекта приводит к декременту счетчика ссылок, доводя его до нуля. Память, на которую указывает временный указатель, освобождается при удалении временного указателя.

Но указатель `x` продолжает указывать на эту (освобожденную) область памяти; теперь `x` — потерянный указатель. Результат попытки использования значения, на которое указывает указатель `x`, непредсказуем.

При связывании указателя `shared_ptr` с простым указателем ответственность за эту память передается указателю `shared_ptr`. Как только ответственность за область памяти встроенного указателя передается указателю `shared_ptr`, больше нельзя использовать встроенный указатель для доступа к памяти, на которую теперь указывает указатель `shared_ptr`.



Опасно использовать встроенный указатель для доступа к объекту, принадлежащему интеллектуальному указателю, поскольку нельзя быть уверенным в том, что этот объект еще не удален.

Другие операции с указателем `shared_ptr`

Класс `shared_ptr` предоставляет также несколько других операций, перечисленных в табл. 12.2 (стр. 576) и табл. 12.3 (стр. 592). Чтобы присвоить новый указатель указателю `shared_ptr`, можно использовать функцию `reset()`:

```
p = new int(1024); // нельзя присвоить обычный указатель
                  // указателю shared_ptr
p.reset(new int(1024)); // ok: p указывает на новый объект
```

Подобно оператору присвоения, функция `reset()` модифицирует счетчики ссылок, а если нужно, удаляет объект, на который указывает указатель `p`. Функцию-член `reset()` зачастую используют вместе с функцией `unique()` для контроля совместного использования объекта несколькими указателями `shared_ptr`. Прежде чем изменять базовый объект, проверяем, является ли владелец единственным. В противном случае перед изменением создается новая копия:

```
if (!p.unique())
    p.reset(new string(*p)); // владелец не один; резервируем новую копию
*p += newVal; // теперь, когда известно, что указатель единственный,
              // можно изменить объект
```

Упражнения раздела 12.1.3

Упражнение 12.10. Укажите, правилен ли следующий вызов функции `process()`, определенной на стр. 593. В противном случае укажите, как его исправить?

```
shared_ptr<int> p(new int(42));
process(shared_ptr<int>(p));
```

Упражнение 12.11. Что будет, если вызвать функцию `process()` следующим образом?

```
process(shared_ptr<int>(p.get()));
```

Упражнение 12.12. Используя объявления указателей `p` и `sp`, объясните каждый из следующих вызовов функции `process()`. Если вызов корректен, объясните, что он делает. Если вызов некорректен, объясните почему:

```
auto p = new int();
auto sp = make_shared<int>();
(a) process(sp);
(b) process(new int());
(c) process(p);
(d) process(shared_ptr<int>(p));
```

Упражнение 12.13. Что будет при выполнении следующего кода?

```
auto sp = make_shared<int>();
auto p = sp.get();
delete p;
```



12.1.4. Интеллектуальные указатели и исключения

В разделе 5.6.2 (стр. 261) упоминалось, что программы, использующие обработку исключений для продолжения работы после того, как произошло исключение, нуждаются в способе правильного освобождения ресурсов в случае исключения. Самый простой из них подразумевает использование интеллектуальных указателей.

При использовании интеллектуального указателя его класс гарантирует освобождение памяти, когда в ней больше нет необходимости, даже при преждевременном выходе из блока:

```
void f()
{
    shared_ptr<int> sp(new int(42)); // зарезервировать новый объект
    // код, передающий исключение, не обрабатываемое в функции f(),
} // shared_ptr освобождает память автоматически по завершении функции
```

При выходе из функции, обычном или в связи с исключением, удаляются все ее локальные объекты. В данном случае указатель `sp` имеет тип `shared_ptr`, поэтому при удалении проверяется его счетчик ссылок. В дан-

ном случае `sp` — единственный указатель на контролируемую им область памяти, поэтому она освобождается в ходе удаления указателя `sp`.

Память, контролируемая непосредственно, напротив, не освобождается автоматически, когда происходит исключение. Если для управления памятью используются встроенные указатели и исключение происходит после оператора `new`, но перед оператором `delete`, то контролируемая память не будет освобождена:

```
void f()
{
    int *ip = new int(42); // динамически зарезервировать новый объект
    // код, передающий исключение, не обрабатываемое в функции f(),
    delete ip; // освобождает память перед выходом
}
```

Если исключение происходит между операторами `new` и `delete` и не обрабатывается в функции `f()`, то освободить эту память никак не получится. Вне функции `f()` нет указателя на эту память, поэтому нет никакого способа освободить ее.



Интеллектуальные указатели и классы без деструкторов

Большинство классов языка C++, включая все библиотечные классы, определяют деструкторы (см. раздел 12.1.1, стр. 578), заботящиеся об удалении используемых объектом ресурсов. Но не все классы таковы. В частности, классы, разработанные для использования и в языке C, и в языке C++, обычно требуют от пользователя явного освобождения всех используемых ресурсов.

Классы, которые резервируют ресурсы, но не определяют деструкторы для их освобождения, подвержены тем же ошибкам, которые возникают при самостоятельном использовании динамической памяти. Довольно просто забыть освободить ресурс. Аналогично, если произойдет исключение после резервирования ресурса, но до его освобождения, программа потеряет его.

Для управления классами без деструкторов зачастую можно использовать те же подходы, что и для управления динамической памятью. Предположим, например, что используется сетевая библиотека, применимая как в языке C, так и в C++. Использующая эту библиотеку программа могла бы содержать такой код:

```
struct destination; // представляет то, с чем установлено соединение
struct connection; // информация для использования соединения
connection connect(destination*); // открывает соединение
void disconnect(connection); // закрывает данное соединение
void f(destination &d /* другие параметры */)
```

```

{
    // получить соединение; не забыть закрывать по завершении
    connection c = connect(&d);
    // использовать соединение
    // если забыть вызывать функцию disconnect() перед выходом из
    // функции f(), то уже не будет никакого способа закрыть соединение c
}

```

Если бы у структуры `connection` был деструктор, то по завершении функции `f()` он закрыл бы соединение автоматически. Однако у нее нет деструктора. Эта проблема почти идентична проблеме предыдущей программы, использовавшей указатель `shared_ptr`, чтобы избежать утечек памяти. Здесь также можно использовать указатель `shared_ptr` для гарантии правильности закрытия соединения.



Использование собственного кода удаления

По умолчанию указатели `shared_ptr` подразумевали, что они указывают на динамическую память. Следовательно, когда указатель `shared_ptr` удаляется, он по умолчанию выполняет оператор `delete` для содержащегося в нем указателя. Чтобы использовать указатель `shared_ptr` для управления соединением `connection`, следует сначала определить функцию, используемую вместо оператора `delete`. Должна быть возможность вызова этой функции удаления (*deleter*) с указателем, хранимым в указателе `shared_ptr`. В данном случае функция удаления должна получать один аргумент типа `connection*`:

```
void end_connection(connection *p) { disconnect(*p); }
```

При создании указателя `shared_ptr` можно передать необязательный аргумент, указывающий на функцию удаления (см. раздел 6.7, стр. 322):

```

void f(destination &d /* другие параметры */)
{
    connection c = connect(&d);
    shared_ptr<connection> p(&c, end_connection);
    // использовать соединение
    // при выходе из функции f(), даже в случае исключения, соединение
    // будет закрыто правильно
}

```

При удалении указателя `p` для хранимого в нем указателя вместо оператора `delete` будет вызвана функция `end_connection()`. Функция `end_connection()`, в свою очередь, вызовет функцию `disconnect()`, гарантируя таким образом закрытие соединения. При нормальном выходе из функции `f()` указатель `p` будет удален в ходе процедуры выхода. Кроме того, указатель `p` будет также удален, а соединение закрыто, если произойдет исключение.

ВНИМАНИЕ! ПРОБЛЕМЫ ИНТЕЛЛЕКТУАЛЬНОГО УКАЗАТЕЛЯ

Интеллектуальные указатели могут обеспечить безопасность и удобство работы с динамически созданной памятью только при правильном использовании. Для этого следует придерживаться ряда соглашений.

- Не используйте значение того же встроенного указателя для инициализации (переустановки) нескольких интеллектуальных указателей.
- Не используйте оператор `delete` для указателя, возвращенного функцией `get()`.
- Не используйте функцию `get()` для инициализации или переустановки другого интеллектуального указателя.
- Используя указатель, возвращенный функцией `get()`, помните, что указатель станет недопустимым после удаления последнего соответствующего интеллектуального указателя.
- Если интеллектуальный указатель используется для управления ресурсом, отличным от области динамической памяти, зарезервированной оператором `new`, не забывайте использовать функцию удаления (раздел 12.1.4, стр. 597 и раздел 12.1.5, стр. 601).

Упражнения раздела 12.1.4

Упражнение 12.14. Напишите собственную версию функции, использующую указатель `shared_ptr` для управления соединением.

Упражнение 12.15. Перепишите первое упражнение так, чтобы использовать лямбда-выражение (см. раздел 10.3.2, стр. 497) вместо функции `end_connection()`.

12.1.5. Класс `unique_ptr`**C++
11**

Указатель `unique_ptr` “владеет” объектом, на который он указывает. В отличие от указателя `shared_ptr`, только один указатель `unique_ptr` может одновременно указывать на данный объект. Объект, на который указывает указатель `unique_ptr`, удаляется при удалении указателя. Список функций, специфических для указателя `unique_ptr`, приведен в табл. 12.4. Функции, общие для обоих указателей, приведены в табл. 12.1 (стр. 575).

В отличие от указателя `shared_ptr`, нет никакой библиотечной функции, подобной функции `make_shared()`, которая возвращала бы указатель `unique_ptr`. Вместо этого определяемый указатель `unique_ptr` связывается

с указателем, возвращенным оператором `new`. Подобно указателю `shared_ptr`, можно использовать прямую форму инициализации:

```
unique_ptr<double> p1; // указатель unique_ptr на тип double
unique_ptr<int> p2(new int(42)); // p2 указывает на int со значением 42
```

Таблица 12.4. Функции указателя `unique_ptr`
(см. также табл. 12.1 на стр. 575)

<code>unique_ptr<T> u1</code> <code>unique_ptr<T, D> u2</code>	Обнуляет указатель <code>unique_ptr</code> , способный указывать на объект типа <code>T</code> . Указатель <code>u1</code> использует для освобождения своего указателя оператор <code>delete</code> ; а указатель <code>u2</code> — вызываемый объект типа <code>D</code>
<code>unique_ptr<T, D> u(d)</code>	Обнуляет указатель <code>unique_ptr</code> , указывающий на объекты типа <code>T</code> . Использует вызываемый объект <code>d</code> типа <code>D</code> вместо оператора <code>delete</code>
<code>u = nullptr</code>	Удаляет объект, на который указывает указатель <code>u</code> ; обнуляет указатель <code>u</code>
<code>u.release()</code>	Прекращает контроль содержимого указателя <code>u</code> ; возвращает содержимое указателя <code>u</code> и обнуляет его
<code>u.reset()</code> <code>u.reset(q)</code> <code>u.reset(nullptr)</code>	Удаляет объект, на который указывает указатель <code>u</code> . Если предоставляется встроенный указатель <code>q</code> , то <code>u</code> будет указывать на его объект. В противном случае указатель <code>u</code> обнуляется

Поскольку указатель `unique_ptr` владеет объектом, на который указывает, он не поддерживает обычного копирования и присвоения:

```
unique_ptr<string> p1(new string("Stegosaurus"));
unique_ptr<string> p2(p1); // ошибка: невозможно копирование unique_ptr
unique_ptr<string> p3;
p3 = p2; // ошибка: невозможно присвоение unique_ptr
```

Хотя указатель `unique_ptr` нельзя ни присвоить, ни скопировать, можно передать собственность от одного (неконстантного) указателя `unique_ptr` другому, вызвав функцию `release()` или `reset()`:

```
// передает собственность от p1 (указывающего на
// строку "Stegosaurus") к p2
unique_ptr<string> p2(p1.release()); // release() обнуляет p1
unique_ptr<string> p3(new string("Trex"));
// передает собственность от p3 к p2
p2.reset(p3.release()); // reset() освобождает память, на которую
// указывал указатель p2
```

Функция-член `release()` возвращает указатель, хранимый в настоящее время в указателе `unique_ptr`, и обнуляет указатель `unique_ptr`. Таким об-

разом, указатель p2 инициализируется указателем, хранимым в указателе p1, а сам указатель p1 становится нулевым.

Функция-член `reset()` получает необязательный указатель и переустанавливает указатель `unique_ptr` на заданный указатель. Если указатель `unique_ptr` не нулевой, то объект, на который он указывает, удаляется. Поэтому вызов функции `reset()` указателя p2 освобождает память, используемую строкой со значением "Stegosaurus", передает содержимое указателя p3 указателю p2 и обнуляет указатель p3.

Вызов функции `release()` нарушает связь между указателем `unique_ptr` и объектом, который он контролирует. Зачастую указатель, возвращенный функцией `release()`, используется для инициализации или присвоения другому интеллектуальному указателю. В этом случае ответственность за управление памятью просто передается от одного интеллектуального указателя другому. Но если другой интеллектуальный указатель не используется для хранения указателя, возвращенного функцией `release()`, то ответственность за освобождения этого ресурса берет на себя программа:

```
p2.release(); // ОШИБКА: p2 не освободит память, и указатель
              // будет потерян
auto p = p2.release(); // ok, но следует не забыть delete(p)
```

Передача и возвращение указателя `unique_ptr`

Из правила, запрещающего копирование указателя `unique_ptr`, есть одно исключение: можно копировать и присваивать те указатели `unique_ptr`, которые предстоит удалить. Наиболее распространенный пример — возвращение указателя `unique_ptr` из функции:

```
unique_ptr<int> clone(int p) {
    // ok: явное создание unique_ptr<int> для int*
    return unique_ptr<int>(new int(p));
}
```

В качестве альтернативы можно также вернуть копию локального объекта:

```
unique_ptr<int> clone(int p) {
    unique_ptr<int> ret(new int (p));
    // ...
    return ret;
}
```

В обоих случаях компилятор знает, что возвращаемый объект будет сейчас удален. В таких случаях компилятор осуществляет специальный вид "копирования", обсуждаемый в разделе 13.6.2 (стр. 676).

СОВМЕСТИМОСТЬ С ПРЕЖНЕЙ ВЕРСИЕЙ: КЛАСС `auto_ptr`

Прежние версии библиотеки включали класс `auto_ptr`, обладавший некоторыми, но не всеми, свойствами указателя `unique_ptr`. В частности, невозможно было хранить указатели `auto_ptr` в контейнере и возвращать их из функции.

Хотя указатель `auto_ptr` все еще присутствует в стандартной библиотеке, вместо него следует использовать указатель `unique_ptr`.

Передача функции удаления указателю `unique_ptr`

Подобно указателю `shared_ptr`, для освобождения объекта, на который указывает указатель `unique_ptr`, по умолчанию используется оператор `delete`. Подобно указателю `shared_ptr`, функцию удаления указателя `unique_ptr` (см. раздел 12.1.4, стр. 597) можно переопределить. Но по причинам, описанным в разделе 16.1.6 (стр. 851), способ применения функции удаления указателем `unique_ptr` отличается от такового у `shared_ptr`.

Переопределение функции удаления указателя `unique_ptr` влияет на тип и способ создания (или переустановки) объектов этого типа. Подобно переопределению оператора сравнения ассоциативного контейнера (см. раздел 11.2.2, стр. 543), тип функции удаления можно предоставить в угловых скобках наряду с типом, на который может указывать указатель `unique_ptr`. При создании или переустановке объекта этого типа предоставляется вызываемый объект определенного типа:

```
// p указывает на объект типа objT и использует объект типа delT
// для его освобождения
// он вызовет объект по имени fcn типа delT
unique_ptr<objT, delT> p (new objT, fcn);
```

В качестве несколько более конкретного примера перепишем программу соединения так, чтобы использовать указатель `unique_ptr` вместо указателя `shared_ptr` следующим образом:

```
void f(destination &d /* другие необходимые параметры */)
{
    connection c = connect(&d); // открыть соединение
    // когда p будет удален, соединение будет закрыто
    unique_ptr<connection, decltype(end_connection)*>
    p(&c, end_connection);
    // использовать соединение
    // по завершении f(), даже при исключении, соединение будет
    // закрыто правильно
}
```

Для определения типа указателя на функцию используется ключевое слово `decltype` (см. раздел 2.5.3, стр. 110). Поскольку выражение `decltype(end_`

connection) возвращает тип функции, следует добавить символ *, указывающий, что используется указатель на этот тип (см. раздел 6.7, стр. 326).

Упражнения раздела 12.1.5

Упражнение 12.16. Компиляторы не всегда предоставляют понятные сообщения об ошибках, если осуществляется попытка скопировать или присвоить указатель `unique_ptr`. Напишите программу, которая содержит эти ошибки, и посмотрите, как компилятор диагностирует их.

Упражнение 12.17. Какие из следующих объявлений указателей `unique_ptr` недопустимы или вероятнее всего приведут к ошибке впоследствии? Объясните проблему каждого из них.

```
int ix = 1024, *pi = &ix, *pi2 = new int(2048);
typedef unique_ptr<int> IntP;
(a) IntP p0(ix);                (b) IntP p1(pi);
(c) IntP p2(pi2);               (d) IntP p3(&ix);
(e) IntP p4(new int(2048));      (f) IntP p5(p2.get());
```

Упражнение 12.18. Почему класс указателя `shared_ptr` не имеет функции-члена `release()`?



12.1.6. Класс `weak_ptr`



Класс `weak_ptr` (табл. 12.5) представляет интеллектуальный указатель, который не контролирует продолжительность существования объекта, на который он указывает. Он только указывает на объект, который контролирует указатель `shared_ptr`. Привязка указателя `weak_ptr` к указателю `shared_ptr` не изменяет счетчик ссылок этого указателя `shared_ptr`. Как только последний указатель `shared_ptr` на этот объект будет удален, удаляется и сам объект. Этот объект будет удален, даже если останется указатель `weak_ptr` на него. Имя `weak_ptr` отражает концепцию “слабого” совместного использования объекта.

Создаваемый указатель `weak_ptr` инициализируется из указателя `shared_ptr`:

```
auto p = make_shared<int>(42);
weak_ptr<int> wp(p); // wp слабо связан с p; счетчик ссылок p неизменен
```

Здесь указатели `wp` и `p` указывают на тот же объект. Поскольку совместное использование слабо, создание указателя `wp` не изменяет счетчик ссылок указателя `p`; это делает возможным удаление объекта, на который указывает указатель `wp`.

Таблица 12.5. Функции указателя `weak_ptr`

<code>weak_ptr<T> w</code>	Обнуляет указатель <code>weak_ptr</code> , способный указывать на объект типа <code>T</code>
<code>weak_ptr<T> w(sp)</code>	Указатель <code>weak_ptr</code> на тот же объект, что и указатель <code>sp</code> типа <code>shared_ptr</code> . Тип <code>T</code> должен быть приводим к типу, на который указывает <code>sp</code>
<code>w = p</code>	Указатель <code>p</code> может иметь тип <code>shared_ptr</code> или <code>weak_ptr</code> . После присвоения <code>w</code> разделяет собственность с указателем <code>p</code>
<code>w.reset()</code>	Обнуляет указатель <code>w</code>
<code>w.use_count()</code>	Возвращает количество указателей <code>shared_ptr</code> , разделяющих собственность с указателем <code>w</code>
<code>w.expired()</code>	Возвращает значение <code>true</code> , когда функция <code>w.use_count()</code> должна вернуть ноль, и значение <code>false</code> в противном случае
<code>w.lock()</code>	Возвращает нулевой указатель <code>shared_ptr</code> , если функция <code>expired()</code> должна вернуть значение <code>true</code> ; в противном случае возвращает указатель <code>shared_ptr</code> на объект, на который указывает указатель <code>w</code>

Поскольку объект может больше не существовать, нельзя использовать указатель `weak_ptr` для непосредственного доступа к его объекту. Для этого следует вызвать функцию `lock()`. Она проверяет существование объекта, на который указывает указатель `weak_ptr`. Если это так, то функция `lock()` возвращает указатель `shared_ptr` на совместно используемый объект. Такой указатель гарантирует существование объекта, на который он указывает, по крайней мере, пока существует этот указатель `shared_ptr`. Рассмотрим пример:

```
if (shared_ptr<int> np = wp.lock()) { // true, если np не нулевой
    // в if, np совместно использует свой объект с p
}
```

Внутренняя часть оператора `if` доступна только в случае истинности вызова функции `lock()`. В операторе `if` использование указателя `np` для доступа к объекту вполне безопасно.

Проверяемый класс указателя

Для того чтобы проиллюстрировать, насколько полезен указатель `weak_ptr`, определим вспомогательный класс указателя для нашего класса `StrBlob`. Класс указателя, назовем его `StrBlobPtr`, будет хранить указатель `weak_ptr` на переменную-член `data` класса `StrBlob`, которым он был инициализирован. Использование указателя `weak_ptr` не влияет на продолжительность существова-

ния вектора, на который указывает данный объект класса `StrBlob`. Но можно воспрепятствовать попытке доступа к вектору, которого больше не существует.

Класс `StrBlobPtr` будет иметь две переменные-члена: указатель `wptr`, который может быть либо нулевым, либо указателем на вектор в объекте класса `StrBlob`; и переменную `curr`, хранящую индекс элемента, который в настоящее время обозначает этот объект. Подобно вспомогательному классу класса `StrBlob`, у класса указателя есть функция-член `check()`, проверяющая безопасность обращения к значению `StrBlobPtr`:

```
// StrBlobPtr передает исключение при попытке доступа к
// несуществующему элементу
class StrBlobPtr {
public:
    StrBlobPtr(): curr(0) { }
    StrBlobPtr(StrBlob &a, size_t sz = 0):
        wptr(a.data), curr(sz) { }
    std::string& deref() const;
    StrBlobPtr& incr(); // префиксная версия
private:
    // check() возвращает shared_ptr на вектор, если проверка успешна
    std::shared_ptr<std::vector<std::string>>
        check(std::size_t, const std::string&) const;
    // хранит weak_ptr, означая возможность удаления основного вектора
    std::weak_ptr<std::vector<std::string>> wptr;
    std::size_t curr; // текущая позиция в пределах массива
};
```

Стандартный конструктор создает нулевой указатель `StrBlobPtr`. Список инициализации его конструктора (см. раздел 7.1.4, стр. 345) явно инициализирует переменную-член `curr` нулем и неявно инициализирует указатель-член `wptr` как нулевой указатель `weak_ptr`. Второй конструктор получает ссылку на `StrBlob` и (необязательно) значение индекса. Этот конструктор инициализирует `wptr` как указатель на вектор данного объекта класса `StrBlob` и инициализирует переменную `curr` значением `sz`. Используем аргумент по умолчанию (см. раздел 6.5.1, стр. 308) для инициализации переменной `curr`, чтобы обозначить первый элемент. Как будет продемонстрировано, ниже параметр `sz` будет использован функцией-членом `end()` класса `StrBlob`.

Следует заметить, что нельзя связать указатель `StrBlobPtr` с константным объектом класса `StrBlob`. Это ограничение следует из того факта, что конструктор получает ссылку на неконстантный объект типа `StrBlob`.

Функция-член `check()` класса `StrBlobPtr` отличается от таковой у класса `StrBlob`, поскольку она должна проверять, существует ли еще вектор, на который он указывает:

```
std::shared_ptr<std::vector<std::string>>
StrBlobPtr::check(std::size_t i, const std::string &msg) const
```

```
{
    auto ret = wptr.lock(); // существует ли еще вектор?
    if (!ret)
        throw std::runtime_error("unbound StrBlobPtr");
    if (i >= ret->size())
        throw std::out_of_range(msg);
    return ret; // в противном случае, вернуть shared_ptr на вектор
}
```

Так как указатель `weak_ptr` не влияет на счетчик ссылок соответствующего указателя `shared_ptr`, вектор, на который указывает `StrBlobPtr`, может быть удален. Если вектора нет, функция `lock()` возвратит нулевой указатель. В таком случае любое обращение к вектору потерпит неудачу и приведет к передаче исключения. В противном случае функция `check()` проверит переданный индекс. Если значение допустимо, функция `check()` возвратит указатель `shared_ptr`, полученный из функции `lock()`.

Операции с указателями

Определение собственных операторов рассматривается в главе 14, а пока определим функции `deref()` и `incr()` для обращения к значению и инкремента указателя класса `StrBlobPtr` соответственно.

Функция-член `deref()` вызывает функцию `check()` для проверки безопасности использования вектора и принадлежности индекса `curr` его диапазону:

```
std::string& StrBlobPtr::deref() const
{
    auto p = check(curr, "dereference past end");
    return (*p)[curr]; // (*p) - вектор, на который указывает этот объект
}
```

Если проверка прошла успешно, то `p` будет указателем типа `shared_ptr` на вектор, на который указывает данный указатель `StrBlobPtr`. Выражение `(*p)[curr]` обращается к значению данного указателя `shared_ptr`, чтобы получить вектор, и использует оператор индексирования для доступа и возвращения элемента по индексу `curr`.

Функция-член `incr()` также вызывает функцию `check()`:

```
// префикс: вернуть ссылку на объект после инкремента
StrBlobPtr& StrBlobPtr::incr()
{
    // если curr уже указывает на элемент после конца контейнера,
    // его инкремент не нужен
    check(curr, "increment past end of StrBlobPtr");
    ++curr; // инкремент текущего состояния
    return *this;
}
```

Безусловно, чтобы получить доступ к переменной-члену `data`, наш класс указателя должен быть дружественным классу `StrBlob` (см. раздел 7.3.4,

стр. 363). Снабдим также класс `StrBlob` функциями `begin()` и `end()`, возвращающими указатель `StrBlobPtr` на себя:

```
// предварительное объявление необходимо для объявления дружественным
// классу StrBlob
class StrBlobPtr;
class StrBlob {
    friend class StrBlobPtr;
    // другие члены, как в разделе 12.1.1 (стр. 582)
    // вернуть указатель StrBlobPtr на первый и следующий
    // после последнего элементы
    StrBlobPtr begin() { return StrBlobPtr(*this); }
    StrBlobPtr end()
        { auto ret = StrBlobPtr(*this, data->size());
          return ret; }
};
```

Упражнения раздела 12.1.6

Упражнение 12.19. Определите собственную версию класса `StrBlobPtr` и модифицируйте класс `StrBlob` соответствующим объявлением дружественным, а также функциями-членами `begin()` и `end()`.

Упражнение 12.20. Напишите программу, которая построчно читает исходный файл в операционной системе класса `StrBlob` и использует указатель `StrBlobPtr` для вывода каждого его элемента.

Упражнение 12.21. Функцию-член `deref()` класса `StrBlobPtr` можно написать следующим образом:

```
std::string& deref() const
{ return (*check(curr, "dereference past end"))[curr]; }
```

Какая версия по-вашему лучше и почему?

Упражнение 12.22. Какие изменения следует внести в класс `StrBlobPtr`, чтобы получить класс, применимый с типом `const StrBlob`? Определите класс по имени `ConstStrBlobPtr`, способный указывать на `const StrBlob`.



12.2. Динамические массивы

Операторы `new` и `delete` резервируют объекты по одному. Некоторым приложениям нужен способ резервировать хранилище для многих объектов сразу. Например, векторы и строки хранят свои элементы в непрерывной памяти и должны резервировать несколько элементов сразу всякий раз, когда контейнеру нужно повторное резервирование (см. раздел 9.4, стр. 457).

Для этого язык и библиотека предоставляют два способа резервирования всего массива объектов. Язык определяет второй вид оператора `new`, резервирующего и инициализирующего массив объектов. Библиотека предоставляет

шаблон класса `allocator`, позволяющий отделять резервирование от инициализации. По причинам, описанным в разделе 12.2.2 (стр. 613), применение класса `allocator` обычно обеспечивает лучшую производительность и более гибкое управление памятью.

У многих (возможно, у большинства) приложений нет никакой непосредственной необходимости в динамических массивах. Когда приложение нуждается в переменном количестве объектов, практически всегда проще, быстрее и безопасней использовать вектор (или другой библиотечный контейнер), как было сделано в классе `StrBlob`. По причинам, описанным в разделе 13.6 (стр. 672), преимущества использования библиотечного контейнера даже более явны по новому стандарту. Библиотеки, поддерживающие новый стандарт, работают существенно быстрее, чем предыдущие версии.



Рекомендуем

Большинство приложений должно использовать библиотечные контейнеры, а не динамически созданные массивы. Использовать контейнер проще, так как меньше вероятность допустить ошибку управления памятью, и, вероятно, он обеспечивает лучшую производительность.

Как уже упоминалось, использующие контейнеры классы могут использовать заданные по умолчанию версии операторов копирования, присвоения и удаления (см. раздел 7.1.5, стр. 347). Классы, резервирующие динамические массивы, должны определить собственные версии этих операторов для управления памятью при копировании, присвоении и удалении объектов.



ВНИМАНИЕ

Не резервируйте динамические массивы в классах, пока не прочитаете главу 13.



12.2.1. Оператор `new` и массивы

Чтобы запросить оператор `new` зарезервировать массив объектов, после имени типа следует указать в квадратных скобках количество резервируемых объектов. В данном случае оператор `new` резервирует требуемое количество объектов и (при успешном резервировании) возвращает указатель на первый из них:

```
// вызов get_size() определит количество резервируемых целых чисел  
int *pia = new int[get_size()]; // pia указывает на первое из них
```

Значение в скобках должно иметь целочисленный тип, но не обязано быть константой.

Для представления типа массива при резервировании можно также использовать псевдоним типа (см. раздел 2.5.1, стр. 106). В данном случае скобки не нужны:

```
typedef int arrT[42]; // arrT - имя типа массива из 42 целых чисел
int *p = new arrT;    // резервирует массив из 42 целых чисел;
                     // p указывает на первый его элемент
```

Здесь оператор `new` резервирует массив целых чисел и возвращает указатель на его первый элемент. Даже при том, что никаких скобок в коде нет, компилятор выполняет это выражение, используя оператор `new[]`. Таким образом, компилятор выполняет это выражение, как будто код был написан так:

```
int *p = new int[42];
```

Резервирование массива возвращает указатель на тип элемента

Хотя обычно память, зарезервированную оператором `new T[]`, называют “динамическим массивом”, это несколько вводит в заблуждение. Когда мы используем оператор `new` для резервирования массива, объект типа массива получен не будет. Вместо этого будет получен указатель на тип элемента массива. Даже если для определения типа массива использовать псевдоним типа, оператор `new` не резервирует объект типа массива. И в данном случае резервируется массив, хотя часть [число] не видима. Даже в этом случае оператор `new` возвращает указатель на тип элемента.

**C++
11** Поскольку зарезервированная память не имеет типа массива, для динамического массива нельзя вызвать функцию `begin()` или `end()` (см. раздел 3.5.3, стр. 168). Для возвращения указателей на первый и следующий после последнего элементы эти функции используют размерность массива (являющуюся частью типа массива). По тем же причинам для обработки элементов так называемого динамического массива нельзя также использовать серийный оператор `for`.



Важно помнить, что у так называемого динамического массива нет типа массива.

Инициализация массива динамически созданных объектов

Зарезервированные оператором `new` объекты (будь то одиночные или их массивы) инициализируются по умолчанию. Для инициализации элементов массива по умолчанию (см. раздел 3.3.1, стр. 143) за размером следует расположить пару круглых скобок:

```
int *pia = new int[10];    // блок из десяти неинициализированных
                          // целых чисел
int *pia2 = new int[10](); // блок из десяти целых чисел,
                          // инициализированных по умолчанию значением 0
string *psa = new string[10]; // блок из десяти пустых строк
string *psa2 = new string[10](); // блок из десяти пустых строк
```


**C++
11** По новому стандарту можно также предоставить в скобках список инициализаторов элементов:

```
// блок из десяти целых чисел, инициализированных соответствующим
// инициализатором
int *pia3 = new int[10]{0,1,2,3,4,5,6,7,8,9};
// блок из десяти строк; первые четыре инициализируются заданными
// инициализаторами, остальные элементы инициализируются значением
// по умолчанию
string *psa3 = new string[10]{"a", "an", "the", string(3,'x')};
```

При списочной инициализации объекта типа встроенного массива (см. раздел 3.5.1, стр. 162) инициализаторы используются для инициализации первых элементов массива. Если инициализаторов меньше, чем элементов, остальные инициализируются значением по умолчанию. Если инициализаторов больше, чем элементов, оператор `new` потерпит неудачу, не зарезервировав ничего. В данном случае оператор `new` передает исключение типа `bad_array_new_length`. Подобно исключению `bad_alloc`, этот тип определен в заголовке `new`.

**C++
11** Хотя для инициализации элементов массива по умолчанию можно использовать пустые круглые скобки, в них нельзя предоставить инициализаторы для элементов. Благодаря этому факту при резервировании массива нельзя использовать ключевое слово `auto` (см. раздел 12.1.2, стр. 585).

Динамическое резервирование пустого массива вполне допустимо

Для определения количества резервируемых объектов можно использовать произвольное выражение:

```
size_t n = get_size(); // get_size() возвращает количество необходимых
                       // элементов
int* p = new int[n];   // резервирует массив для содержания элементов
for (int* q = p; q != p + n; ++q)
    /* обработка массива */;
```

Возникает интересный вопрос: что будет, если функция `get_size()` возвратит значение 0? Этот код сработает прекрасно. Вызов функции `new[n]` при `n` равном 0 вполне допустим, даже при том, что нельзя создать переменную типа массива размером 0:

```
char arr[0]; // ошибка: нельзя определить массив нулевой длины
char *cp = new char[0]; // ok: но обращение к значению cp件不可能
```

При использовании оператора `new` для резервирования массива нулевого размера он возвращает допустимый, а не нулевой указатель. Этот указатель гарантированно будет отличен от любого другого указателя, возвращенного оператором `new`. Он будет подобен указателю на элемент после конца (см. раздел 3.5.3, стр. 169) для нулевого элемента массива. Этот указатель мож-

но использовать теми способами, которыми используется итератор после конца. Его можно сравнивать в цикле, как выше. К нему можно добавить ноль (или вычесть ноль), такой указатель можно вычесть из себя, получив в результате ноль. К значению такого указателя нельзя обратиться, в конце концов, он не указывает на элемент.

В гипотетическом цикле, если функция `get_size()` возвращает 0, то `n` также равно 0. Вызов оператора `new` зарезервирует ноль объектов. Условие оператора `for` будет ложно (`p` равно `q + n`, поскольку `n` равно 0). Таким образом, тело цикла не выполняется.

Освобождение динамических массивов

Для освобождения динамического массива используется специальная форма оператора `delete`, имеющая пустую пару квадратных скобок:

```
delete p;           // p должен указывать на динамически созданный объект или
                    // быть нулевым
delete [] pa;       // pa должен указывать на динамически созданный объект или
                    // быть нулевым
```

Второй оператор удаляет элементы массива, на который указывает `pa`, и освобождает соответствующую память. Элементы массива удаляются в обратном порядке. Таким образом, последний элемент удаляется первым, затем предпоследний и т.д.

При применении оператора `delete` к указателю на массив пустая пара квадратных скобок необходима: она указывает компилятору, что указатель содержит адрес первого элемента массива объектов. Если пропустить скобки оператора `delete` для указателя на массив (или предоставить их, передав оператору `delete` указатель на объект), то его поведение будет непредсказуемо.

Напомним, что при использовании псевдонима типа, определяющего тип массива, можно зарезервировать массив без использования `[]` в операторе `new`. Но даже в этом случае нужно использовать скобки при удалении указателя на этот массив:

```
typedef int arrT[42]; // arrT имя типа массив из 42 целых чисел
int *p = new arrT;   // резервирует массив из 42 целых чисел; p указывает
                    // на первый элемент
delete [] p;          // скобки необходимы, поскольку был
                    // зарезервирован массив
```

Несмотря на внешний вид, указатель `p` указывает на первый элемент массива объектов, а не на отдельный объект типа `arrT`. Таким образом, при удалении указателя `p` следует использовать `[]`.



ВНИМАНИЕ

Компилятор вряд ли предупредит нас, если забыть скобки при удалении указателя на массив или использовать их при удалении указателя на объект. Программа будет выполняться с ошибкой, не предупреждая о ее причине.

Интеллектуальные указатели и динамические массивы

Библиотека предоставляет версию указателя `unique_ptr`, способную контролировать массивы, зарезервированные оператором `new`. Чтобы использовать указатель `unique_ptr` для управления динамическим массивом, после типа объекта следует расположить пару пустых скобок:

```
// up указывает на массив из десяти неинициализированных целых чисел
unique_ptr<int[]> up(new int[10]);
up.release(); // автоматически использует оператор delete[] для
              // удаления указателя
```

Скобки в спецификаторе типа (`<int[]>`) указывают, что указатель `up` указывает не на тип `int`, а на массив целых чисел. Поскольку указатель `up` указывает на массив, при удалении его указателя автоматически используется оператор `delete[]`.

Указатели `unique_ptr` на массивы предоставляют несколько иные функции, чем те, которые использовались в разделе 12.1.5 (стр. 598). Эти функции описаны в табл. 12.6. Когда указатель `unique_ptr` указывает на массив, нельзя использовать точечный и стрелочный операторы доступа к элементам. В конце концов, указатель `unique_ptr` указывает на массив, а не на объект, поэтому эти операторы были бы бессмысленны. С другой стороны, когда указатель `unique_ptr` указывает на массив, для доступа к его элементам можно использовать оператор индексирования:

```
for (size_t i = 0; i != 10; ++i)
    up[i] = i; // присвоить новое значение каждому из элементов
```

Таблица 12.6. Функции указателя `unique_ptr` на массив

Операторы доступа к элементам (точка и стрелка) не поддерживаются указателями <code>unique_ptr</code> на массивы. Другие его функции неизменны	
<code>unique_ptr<T[]> u</code>	<code>u</code> может указывать на динамически созданный массив типа <code>T</code>
<code>unique_ptr<T[]> u(p)</code>	<code>u</code> указывает на динамически созданный массив, на который указывает встроенный указатель <code>p</code> . Тип указателя <code>p</code> должен допускать приведение к типу <code>T*</code> (см. раздел 4.11.2, стр. 221). Выражение <code>u[i]</code> возвратит объект в позиции <code>i</code> массива, которым владеет указатель <code>u</code> . <code>u</code> должен быть указателем на массив

В отличие от указателя `unique_ptr`, указатель `shared_ptr` не оказывает прямой поддержки управлению динамическим массивом. Если необходимо использовать указатель `shared_ptr` для управления динамическим массивом, следует предоставить собственную функцию удаления:

```
// чтобы использовать указатель shared_ptr, нужно предоставить
// функцию удаления
shared_ptr<int> sp(new int[10], [](int *p) { delete[] p; });
sp.reset(); // использует предоставленное лямбда-выражение, которое в
// свою очередь использует оператор delete[] для освобождения массива
```

Здесь лямбда-выражение (см. раздел 10.3.2, стр. 497), использующее оператор `delete[]`, передается как функция удаления.

Если не предоставить функции удаления, результат выполнения этого кода непредсказуем. По умолчанию указатель `shared_ptr` использует оператор `delete` для удаления объекта, на который он указывает. Если объект является динамическим массивом, то при использовании оператора `delete` возникнут те же проблемы, что и при пропуске `[]`, когда удаляется указатель на динамический массив (см. раздел 12.2.1, стр. 611).

Поскольку указатель `shared_ptr` не поддерживает прямой доступ к массиву, для обращения к его элементам применяется следующий код:

```
// shared_ptrs не имеет оператора индексирования и не поддерживает
// арифметических действий с указателями
for (size_t i = 0; i != 10; ++i)
    *(sp.get() + i) = i; // для доступа к встроенному указателю
                        // используется функция get()
```

Указатель `shared_ptr` не имеет оператора индексирования, а типы интеллектуальных указателей не поддерживают арифметических действий с указателями. В результате для доступа к элементам массива следует использовать функцию `get()`, возвращающую встроенный указатель, который можно затем использовать обычным способом.

Упражнения раздела 12.2.1

Упражнение 12.23. Напишите программу, конкатенирующую два строковых литерала и помещающую результат в динамически созданный массив символов. Напишите программу, конкатенирующую две строки библиотечного типа `string`, имеющих те же значения, что и строковые литералы, используемые в первой программе.

Упражнение 12.24. Напишите программу, которая читает строку со стандартного устройства ввода в динамически созданный символьный массив. Объясните, как программа обеспечивает ввод данных переменного размера. Проверьте свою программу, введя строку, размер которой превышает длину зарезервированного массива.

Упражнение 12.25. С учетом следующего оператора `new`, как будет удаляться указатель `pa`?

```
int *pa = new int[10];
```

12.2.2. Класс `allocator`

Важный аспект, ограничивающий гибкость оператора `new`, заключается в том, что он объединяет резервирование памяти с созданием объекта (объектов) в этой памяти. Точно так же оператор `delete` объединяет удаление объекта с освобождением занимаемой им памяти. Обычно объединение инициализации с резервированием — это именно то, что и нужно при резервировании одиночного объекта. В этом случае почти наверняка известно значение, которое должен иметь объект.

Когда резервируется блок памяти, обычно в нем планируется создавать объекты по мере необходимости. В таком случае желательно было бы отделить резервирование памяти от создания объектов. Это позволит резервировать память в больших объемах, а дополнительные затраты на создание объектов нести только тогда, когда это фактически необходимо.

Зачастую объединение резервирования и создания оказывается расточительным. Например:

```
string *const p = new string[n]; // создает n пустых строк
string s;
string *q = p; // q указывает на первую строку
while (cin >> s && q != p + n)
    *q++ = s; // присваивает новое значение *q
const size_t size = q - p; // запомнить количество прочитанных строк
// использовать массив
delete[] p; // p указывает на массив; не забыть использовать delete[]
```

Этот оператор `new` резервирует и инициализирует `n` строк. Но `n` строк может не понадобиться, — вполне может хватить меньшего количества строк. В результате, возможно, были созданы объекты, которые никогда не будут использованы. Кроме того, тем из объектов, которые действительно используются, новые значения присваиваются немедленно, поверх только что инициализированных строк. Используемые элементы записываются дважды: сначала, когда им присваивается значение по умолчанию, а затем, когда им присваивается значение.

Еще важнее то, что классы без стандартных конструкторов не могут быть динамически созданы как массив.

Класс `allocator` и специальные алгоритмы

Библиотечный класс `allocator`, определенный в заголовке `memory`, позволяет отделить резервирование от создания. Он обеспечивает не типизированное резервирование свободной области памяти. Операции, поддерживаемые классом `allocator`, приведены в табл. 12.7. Операции с классом `allocator` описаны в этом разделе, а типичный пример его использования — в разделе 13.5 (стр. 664).

Подобно типу `vector`, тип `allocator` является шаблоном (см. раздел 3.3, стр. 141). Чтобы определить экземпляр класса `allocator`, следует указать тип объектов, которые он сможет резервировать. Когда объект `allocator` резервирует память, он обеспечивает непрерывное хранилище соответствующего размера для содержания объектов заданного типа:

```
allocator<string> alloc;           // объект, способный резервировать строки
auto const p = alloc.allocate(n); // резервирует n незаполненных строк
```

Этот вызов функции `allocate()` резервирует память для `n` строк.

Таблица 12.7. Стандартный класс `allocator` и специальные алгоритмы

<code>allocator<T> a</code>	Определяет объект <code>a</code> класса <code>allocator</code> , способный резервировать память для объектов типа <code>T</code>
<code>a.allocate(n)</code>	Резервирует пустую область памяти для содержания <code>n</code> объектов типа <code>T</code>
<code>a.deallocate(p, n)</code>	Освобождает область памяти, содержащую <code>n</code> объектов типа <code>T</code> , начиная с адреса в указателе <code>p</code> типа <code>T*</code> . Указатель <code>p</code> должен быть ранее возвращен функцией <code>allocate()</code> , а размер <code>n</code> — соответствовать запрошенному при создании указателя <code>p</code> . Функцию <code>destroy()</code> следует выполнить для всех объектов, созданных в этой памяти, прежде, чем вызвать функцию <code>deallocate()</code>
<code>a.construct(p, args)</code>	Указатель <code>p</code> на тип <code>T</code> должен указывать на незаполненную область памяти; аргументы <code>args</code> передаются конструктору типа <code>T</code> , используемому для создания объекта в памяти, на которую указывает указатель <code>p</code>
<code>a.destroy(p)</code>	Выполняет деструктор (см. раздел 12.1.1, стр. 578) для объекта, на который указывает указатель <code>p</code> типа <code>T*</code>

Класс `allocator` резервирует незаполненную память



Память, которую резервирует объект класса `allocator`, не заполнена. Эта область памяти используется при создании объектов. В новой библиотеке функция-член `construct()` получает указатель и любое количество дополнительных аргументов; она создает объекты в заданной области памяти. Для инициализации создаваемого объекта используются дополнительные аргументы. Подобно аргументам функции `make_shared()` (см. раздел 12.1.1, стр. 576), эти дополнительные аргументы должны быть допустимыми инициализаторами объекта создаваемого типа. В частности, если типом объекта является класс, эти аргументы должны соответствовать конструктору этого класса:

```

auto q = p; // q указывает на следующий элемент после последнего
            // созданного
alloc.construct(q++);           // *q - пустая строка
alloc.construct(q++, 10, 'c'); // *q - ccccccccc
alloc.construct(q++, "hi");     // *q - hi!

```

В прежних версиях библиотеки функция `construct()` получала только два аргумента: указатель для создаваемого объекта и значение его типа. В результате можно было только скопировать объект в незаполненную область, но никакой другой конструктор этого типа использовать было нельзя.

Использование незаполненной области памяти, в которой еще не был создан объект, является ошибкой:

```

cout << *p << endl; // ok: использует оператор вывода класса string
cout << *q << endl; // ошибка: q указывает на незаполненную память!

```



ВНИМАНИЕ

Чтобы использовать память, возвращенную функцией `allocate()`, в ней следует создать объекты. Результат использования незаполненной памяти другими способами непредсказуем.

По завершении использования объектов следует удалить ранее созданные элементы. Для этого следует вызвать функцию `destroy()` каждого созданного элемента. Функция `destroy()` получает указатель и запускает деструктор (см. раздел 12.1.1, стр. 578) указанного объекта:

```

while (q != p)
    alloc.destroy(--q); // освободить фактически зарезервированные строки

```

В начале цикла `q` указывает на следующий элемент после последнего заполненного. Перед вызовом функции `destroy()` осуществляется декремент указателя `q`. Таким образом, при первом вызове функции `destroy()` указатель `q` указывает на последний созданный элемент. Первый элемент удаляется на последней итерации, после которой `q` станет равен `p` и цикл закончится.



ВНИМАНИЕ

Удалять можно только те элементы, которые были фактически созданы.

Как только элементы удалены, память можно повторно использовать для содержания других строк или вернуть их операционной системе. Для освобождения памяти используется функция `deallocate()`:

```

alloc.deallocate(p, n);

```

Указатель, передаваемый функции `deallocate()`, не может быть нулевым; он должен указывать на область памяти, зарезервированной функцией `allocate()`. Кроме того, переданный ей аргумент размера должен совпадать с размером, использованным при вызове функции `allocate()`, зарезервировавшим область памяти, на которую указывает указатель.

Алгоритмы копирования и заполнения неинициализированной памяти

В дополнение к классу `allocator` библиотека предоставляет два алгоритма, способных создавать объекты в неинициализированной памяти. Эти функции описаны в табл. 12.8 и определены в заголовке `memory`.

Таблица 12.8. Алгоритмы, связанные с классом `allocator`

Эти функции создают элементы по назначению, а не присваивают их	
<code>uninitialized_copy(b, e, b2)</code>	Копирует элементы из исходного диапазона, обозначенного итераторами <code>b</code> и <code>e</code> , в незаполненную память, обозначенную итератором <code>b2</code> . Память, обозначенная итератором <code>b2</code> , должна быть достаточно велика для содержания копии элементов из исходного диапазона
<code>uninitialized_copy_n(b, n, b2)</code>	Копирует <code>n</code> элементов, начиная с обозначенного итератором <code>b</code> в незаполненную память, начиная с позиции <code>b2</code>
<code>uninitialized_fill(b, e, t)</code>	Создает объекты в диапазоне незаполненной памяти, обозначенной итераторами <code>b</code> и <code>e</code> как копию <code>t</code>
<code>uninitialized_fill_n(b, n, t)</code>	Создает <code>n</code> объектов, начиная с <code>b</code> . Итератор <code>b</code> должен обозначать незаполненную память достаточного размера для содержания заданного количества объектов

Предположим, например, что имеется вектор целых чисел, который необходимо скопировать в динамическую память. Память будет резервироваться дважды для каждого целого числа в векторе. Первую половину вновь зарезервированной памяти заполним копиями элементов из исходного вектора. Элементы второй половины заполним заданным значением:

```
// зарезервировать вдвое больше элементов, чем хранения в vi
auto p = alloc.allocate(vi.size() * 2);
// создать элементы, начиная с p как копии элементов в vi
auto q = uninitialized_copy(vi.begin(), vi.end(), p);
// инициализировать остальные элементы значением 42
uninitialized_fill_n(q, vi.size(), 42);
```

Подобно алгоритму `copy()` (см. раздел 10.2.2, стр. 489), алгоритм `uninitialized_copy()` получает три итератора. Первые два обозначают исходную последовательность, а третий обозначает получателя, в который будут скопированы эти элементы. Итератор назначения, переданный алгоритму

`uninitialized_copy()`, должен обозначить незаполненную память. В отличие от алгоритма `copy()`, алгоритм `uninitialized_copy()` создает элементы в своем получателе.

Подобно алгоритму `copy()`, алгоритм `uninitialized_copy()` возвращает (приращенный) итератор назначения. Таким образом, вызов функции `uninitialized_copy()` возвращает указатель на следующий элемент после последнего заполненного. В данном примере этот указатель сохраняется в переменной `q`, передаваемой функции `uninitialized_fill_n()`. Эта функция, как и функция `fill_n()` (см. раздел 10.2.2, стр. 488), получает указатель на получателя, количество и значение. Она создает заданное количество объектов из заданного значения в позиции, начиная с заданной получателем.

Упражнения раздела 12.2.2

Упражнение 12.26. Перепишите программу на стр. 613, используя класс `allocator`.



12.3. Использование библиотеки: программа запроса текста

Для завершения обсуждения библиотеки реализуем простую программу текстового запроса. Она позволит пользователю искать в заданном файле слова, которые могли встречаться в нем. Результатом запроса будет количество экземпляров слова и список строк, в которых оно присутствует. Если слово встречается несколько раз в той же строке, то оно отображается только однажды. Строки отображаются в порядке возрастания, т.е. строка номер 7 отображается перед строкой номер 9 и т.д.

Например, прочитав файл, содержащий начало этой главы и запустив поиск слова `element`, программа должна создать следующий вывод:

```
element occurs 112 times
(line 36) A set element contains only a key;
(line 158) operator creates a new element
(line 160) Regardless of whether the element
(line 168) When we fetch an element from a map, we
(line 214) If the element is not found, find returns
```

Далее следует примерно 100 строк, также содержащих слово `element`.



12.3.1. Проект программы

Наилучший способ начать проект программы — это составить перечень ее функциональных возможностей. Зная, какие именно функции необходимо обеспечить, значительно легче разобраться, какие именно структуры данных

понадобятся. Итак, начнем с требований, которым должна удовлетворять разрабатываемая программа.

- Читая ввод, программа должна запоминать строку (строки), в которой присутствует искомое слово. Следовательно, программа должна читать ввод построчно и разделять прочитанные строки на отдельные слова
- При создании вывода программа должна:
 - получать номера строк, содержащих искомое слово;
 - нумеровать строки в порядке возрастания без дубликатов;
 - отображать текст исходного файла по заданному номеру строки.

Эти требования можно выполнить с помощью библиотечных средств.

- Для хранения копии всего входного файла используем вектор `vector<string>`. Каждая строка входного файла станет элементом этого вектора. При необходимости вывода строку можно будет выбрать, используя ее номер как индекс.
- Для разделения строки на слова используем строковый поток `istringstream` (см. раздел 8.3, стр. 413).
- Для хранения номеров строк, в которых присутствует искомое слово, используем контейнер `set`. Это гарантирует, что каждая строка будет присутствовать только однажды, а номера строки будут храниться в порядке возрастания.
- Для связи каждого слова с набором номеров строк, в которых присутствует искомое слово, используем контейнер `map`. Это позволит выбрать соответствующий набор для каждого заданного слова.

По рассматриваемым вскоре причинам в решении будет также использован указатель `shared_ptr`.

Структуры данных

Несмотря на то что программу можно было бы написать, используя контейнеры `vector`, `set` и `map` непосредственно, полезней определить более абстрактное решение. Для начала разработаем класс для содержания входного файла, чтобы упростить запрос. Этот класс, `TextQuery`, будет содержать вектор и карту. Вектор будет содержать текст входного файла, а карта ассоциировать каждое слово в этом файле с набором номеров строк, в которых присутствует искомое слово. У этого класса будет конструктор, читающий заданный входной файл, и функция, обрабатывающая запросы.

Работа функции обработки запроса довольно проста: она просматривает свою карту в поисках искомого слова. Трудней всего решить, что должна возвращать функция запроса. Если известно, что слово найдено, необходимо вы-

яснить, сколько раз оно встретилось, в строках с какими номерами и каков текст каждой строки с этими номерами.

Проще всего вернуть все эти данные, определив второй класс, назовем его `QueryResult`, который и будет содержать результаты запроса. У этого класса будет функция `print()`, выводящая результаты, хранимые в объекте класса `QueryResult`.

Совместное использование данных классами

Класс `QueryResult` предназначен для представления результатов запроса. Эти результаты включают набор номеров строк, связанных с искомым словом, и текст соответствующих строк из входного файла. Эти данные хранятся в объектах типа `TextQuery`.

Поскольку данные, необходимые объекту класса `QueryResult`, хранятся в объекте класса `TextQuery`, необходимо решить, как получить к ним доступ. Можно было бы скопировать набор номеров строк, но это может оказаться слишком дорого. Кроме того, копировать вектор не хотелось бы потому, что это повлечет за собой копирование всего файла для вывода лишь его части (как обычно и бывает).

Избежать копирования можно, возвратив итераторы (или указатели) на содержимое объекта `TextQuery`. Но с этим подходом связана проблема: что если объект класса `TextQuery` будет удален до объекта класса `QueryResult`? В этом случае объект класса `QueryResult` ссылался бы на данные в больше не существующем объекте.

Это требует синхронизации продолжительности существования объекта класса `QueryResult` с объектом класса `TextQuery`, результаты которого он представляет. Таким образом, эти два класса концептуально “совместно используют” данные. Для отражения совместного использования этих структур данных используем указатели `shared_ptr` (см. раздел 12.1.1, стр. 575).

Применение класса `TextQuery`

При разработке класса может быть полезно написать использующие его программы, прежде чем фактически реализовать его члены. Таким образом можно выяснить, какие функции ему необходимы. Например, следующая программа использует проектируемые классы `TextQuery` и `QueryResult`. Эта функция получает поток класса `ifstream` для подлежащего обработке файла и взаимодействует с пользователем, выводя результаты по заданному слову:

```
void runQueries(ifstream &infile)
{
    // infile - поток ifstream для входного файла
    TextQuery tq(infile); // хранит файл и строит карту запроса
    // цикл взаимодействия с пользователем: приглашение ввода искомого
    // слова и вывод результатов
    while (true) {
```

```

cout << "enter word to look for, or q to quit: ";
string s;
// остановиться по достижении конца файла или при встрече
// символа 'q' во вводе
if (!(cin >> s) || s == "q") break;
// выполнить запрос и вывести результат
print(cout, tq.query(s)) << endl;
}
}

```

Начнем с инициализации объекта `tq` класса `TextQuery` данными из переданного потока `ifstream`. Конструктор `TextQuery()` читает файл в свой вектор и создает карту, связывающую слова из ввода с номерами строк, в которых они присутствуют.

Цикл `while` (непрерывно) запрашивает у пользователя искомое слово и выводит полученные результаты. Условие цикла проверяет литерал `true` (см. раздел 2.1.3, стр. 73), поэтому оно всегда истинно. Для выхода из цикла используется оператор `break` (см. раздел 5.5.1, стр. 254) в операторе `if`. Он проверяет успешность чтения. Если оно успешно, проверяется также, не ввел ли пользователь символ "q", желая завершить работу. Получив искомое слово, запрашиваем его поиск у объекта `tq`, а затем вызываем функцию `print()` для вывода результата поиска.

Упражнения раздела 12.3.1

Упражнение 12.27. Классы `TextQuery` и `QueryResult` используют только те возможности, которые уже были описаны ранее. Не заглядывая вперед, напишите собственные версии этих классов.

Упражнение 12.28. Напишите программу, реализующую текстовые запросы, не определяя классы управления данными. Программа должна получать файл и взаимодействовать с пользователем, запрашивая слова, искомые в этом файле. Используйте контейнеры `vector`, `map` и `set` для хранения данных из файла и создания результатов запросов.

Упражнение 12.29. Перепишите цикл взаимодействия с пользователем, используя цикл `do while` (см. раздел 5.4.4, стр. 253). Объясните, какая версия предпочтительней и почему.



12.3.2. Определение классов программы запросов

Начнем с определения класса `TextQuery`. Пользователь создает объекты этого класса, предоставляя поток `istream` для чтения входного файла. Этот класс предоставляет также функцию `query()`, которая получает строку и возвращает объект класса `QueryResult`, представляющий строки, в которых присутствует искомое слово.

Переменные-члены класса должны учитывать совместное использование с объектами класса `QueryResult`. Класс `QueryResult` совместно использует вектор, представляющий входной файл и наборы, содержащие номера строк, связанные с каждым словом во вводе. Следовательно, у нашего класса есть две переменные-члена: указатель `shared_ptr` на динамически созданный вектор, содержащий входной файл, а также карта строк и указателей `shared_ptr<set>`. Карта ассоциирует каждое слово в файле с динамически созданным набором, содержащим номера строк, в которых присутствует это слово.

Чтобы сделать код немного понятней, определим также тип-член (см. раздел 7.3.1, стр. 353) для обращения к номерам строк, которые являются индексами вектора строк:

```
class QueryResult; // объявление необходимого типа возвращаемого значения
                  // функции запроса
class TextQuery {
public:
    using line_no = std::vector<std::string>::size_type;
    TextQuery(std::ifstream&);
    QueryResult query(const std::string&) const;
private:
    std::shared_ptr<std::vector<std::string>> file; // исходный файл
    // сопоставить каждое слово с набором строк, в которых присутствует
    // это слово
    std::map<std::string,
              std::shared_ptr<std::set<line_no>>> wm;
};
```

Самая трудная часть этого кода — разобраться в именах классов. Как обычно, для кода из файла заголовка применяется часть `std::`, указывающая имя библиотеки (м. раздел 3.1, стр. 125). Но в данном случае частое повторение имени `std::` делает код немного менее понятным для чтения. Рассмотрим пример:

```
std::map<std::string, std::shared_ptr<std::set<line_no>>> wm;
```

Его будет проще понять, переписав так:

```
map<string, shared_ptr<set<line_no>>> wm;
```

Конструктор `TextQuery()`

Конструктор `TextQuery()` получает поток `ifstream`, позволяющий читать строки по одной:

```
// прочитать входной файл, создать карту строк и их номеров
TextQuery::TextQuery(ifstream &is): file(new vector<string>)
{
    string text;
    while (getline(is, text)) { // для каждой строки в файле
        file->push_back(text); // запомнить эту строку текста
        int n = file->size() - 1; // номер текущий строки
```

```

istreamstream line(text); // разделить строку на слова
string word;
while (line >> word) { // для каждого слова в этой строке
    // если слова еще нет в wm, индексация добавляет новый
    // элемент
    auto &lines = wm[word]; // lines - это shared_ptr
    if (!lines) // этот указатель - вначале нулевой, когда
        // встречается слово
        lines.reset(new set<line_no>); // резервирует новый набор
    lines->insert(n); // вставить номер этой строки
}
}
}

```

Список инициализации конструктора резервирует новый вектор для содержания текста из входного файла. Функция `getline()` используется для чтения из файла по одной строке за раз и их помещения в вектор. Поскольку `file` — это указатель `shared_ptr`, используем оператор `->` для обращения к его значению, чтобы вызвать функцию `push_back()` для того элемента вектора, на который указывает указатель `file`.

Затем поток `istreamstream` (см. раздел 8.3, стр. 413) используется для обработки каждого слова только что прочитанной строки. Внутренний цикл `while` использует оператор ввода класса `istreamstream` для чтения каждого слова текущей строки в строку `word`. В цикле `while` используется оператор индексирования карты для доступа к связанному со словом указателю `shared_ptr<set>` и связи ссылки `lines` с этим указателем. Обратите внимание, что `lines` — это ссылка, поэтому внесенные в нее изменения будут сделаны с элементом карты `wm`.

Если слова еще нет в карте, оператор индексирования добавит строку `word` в карту `wm` (см. раздел 11.3.4, стр. 555). Ассоциируемый со строкой `word` элемент инициализирован значением по умолчанию. Это значит, что ссылка `lines` будет нулевой, если оператор индексирования добавит строку `word` в карту `wm`. Если ссылка `lines` будет нулевой, резервируем новый набор и вызываем функцию `reset()` для обновления указателя `shared_ptr`, на который ссылается ссылка `lines`, чтобы он указывал на этот только что созданный набор.

Независимо от того, был ли создан новый набор, происходит вызов функции `insert()`, добавляющей текущий номер строки. Поскольку `lines` — это ссылка, вызов функции `insert()` добавляет элемент в набор карты `wm`. Если данное слово встречается несколько раз в той же строке, вызов функции `insert()` не делает ничего.

Класс `QueryResult`

Класс `QueryResult` обладает тремя переменными-членами: строка, представляющая слово, указатель `shared_ptr` на вектор, содержащий входной файл; и указатель `shared_ptr` на набор номеров строк, в которых присутст-

ует это слово. Его единственная функция-член — конструктор, инициализирующий эти три члена:

```
class QueryResult {
friend std::ostream& print(std::ostream&, const QueryResult&);
public:
    QueryResult(std::string s,
                std::shared_ptr<std::set<line_no>> p,
                std::shared_ptr<std::vector<std::string>> f):
        sought(s), lines(p), file(f) { }
private:
    std::string sought; // слово, представляющее запрос
    std::shared_ptr<std::set<line_no>> lines; // номера строк
    std::shared_ptr<std::vector<std::string>> file; // входной файл
};
```

Единственная задача конструктора — сохранить свои аргументы в соответствующих переменных-членах, что он и делает в списке инициализации конструктора (см. раздел 7.1.4, стр. 345).

Функция `query()`

Функция `query()` получает строку, которую она использует для поиска соответствующего набора номеров строк в карте. Если строка найдена, функция `query()` создает объект класса `QueryResult` из заданной строки, переменной-члена `file` класса `TextQuery` и набора, извлеченного из карты `wm`.

Единственный вопрос: что следует вернуть, если заданная строка не найдена? В данном случае никакого набора возвращено не будет. Решим эту проблему, определив локальный статический объект, являющийся указателем `shared_ptr` на пустой набор номеров строк. Когда слово не найдено, возвратим копию этого указателя `shared_ptr`:

```
QueryResult
TextQuery::query(const string &sought) const
{
    // вернуть указатель на этот набор, если искомое слово не найдено
    static shared_ptr<set<line_no>> nodata(new set<line_no>);
    // использовать find() но не индексировать, чтобы избежать
    // добавления слова в карту wm!
    auto loc = wm.find(sought);
    if (loc == wm.end())
        return QueryResult(sought, nodata, file); // не найдено
    else
        return QueryResult(sought, loc->second, file);
}
```

Вывод результатов

Функция `print()` выводит заданный объект класса `QueryResult` в заданный поток:

```
ostream &print(ostream & os, const QueryResult &qr)
{
    // если слово найдено, вывести количество и все вхождения
    os << qr.sought << " occurs " << qr.lines->size() << " "
    << make_plural(qr.lines->size(), "time", "s") << endl;
    // вывести каждую строку, в которой присутствует слово
    for (auto num : *qr.lines) // для каждого элемента в наборе
        // не путать пользователя с номерами строк, начинающимися с 0
        os << "\t(line " << num + 1 << " "
        << *(qr.file->begin() + num) << endl;
    return os;
}
```

Для отчета о количестве найденных соответствий используем функцию `size()` набора, на который ссылается `qr.lines`. Поскольку этот набор контролируется указателем `shared_ptr`, следует помнить об обращении к значению `lines`. Чтобы вывести слово `time` или `times`, в зависимости от того, равен ли размер 1, используем функцию `make_plural()` (см. раздел 6.3.2, стр. 294).

Цикл `for` перебирает набор, на который ссылается `lines`. Тело цикла `for` выводит номер строки, откорректированный так, как привычно человеку. Числа в наборе являются индексами элементов в векторе, их нумерация начинается с нуля. Но большинство пользователей привыкли к тому, что первая строка имеет номер 1, поэтому будем систематически добавлять 1 к номерам строк, чтобы отображать их в общепринятой форме.

Используем номер строки для выбора строк из вектора, на который указывает указатель-член `file`. Помните, что при добавлении числа к итератору будет получен элемент на столько же элементов далее (см. раздел 3.4.2, стр. 159). Таким образом, часть `file->begin() + num` дает номер элемента от начала вектора, на который указывает `file`.

Обратите внимание: эта функция правильно обрабатывает случай, когда слово не найдено. В данном случае набор будет пуст. Первый оператор вывода заметит, что слово встретилось нуль раз. Поскольку `*res.lines` пуст, цикл `for` не выполнится ни разу.

Упражнения раздела 12.3.2

Упражнение 12.30. Определите собственные версии классов `TextQuery` и `QueryResult`, а также выполните функцию `runQueries()` из раздела 12.3.1 (стр. 619).

Упражнение 12.31. Что будет, если для хранения номеров строк использовать вектор вместо набора? Какой подход лучше? Почему?

Упражнение 12.32. Перепишите классы `TextQuery` и `QueryResult` так, чтобы для хранения входного файла вместо вектора `vector<string>` использовался класс `StrBlob`.

Упражнение 12.33. В главе 15 программа запроса будет дополнена, и классу `QueryResult` понадобятся дополнительные члены. Добавьте функции-члены по имени `begin()` и `end()`, возвращающие итераторы для набора номеров строк, возвращенных данным запросом, и функцию-член `get_file()`, возвращающую указатель `shared_ptr` на файл в объекте `QueryResult`.

Резюме

В языке C++ для резервирования памяти используется оператор `new`, а для освобождения — оператор `delete`. Библиотека определяет также класс `allocator`, чтобы резервировать пустые блоки динамической памяти.

Резервирующие динамическую память программы отвечают за ее освобождение. Освобождение динамической памяти — богатейший источник ошибок: память может быть не освобождена никогда или может быть освобождена, когда еще есть указатели на нее. Новая библиотека определяет классы интеллектуальных указателей (`shared_ptr`, `unique_ptr` и `weak_ptr`), делающие работу с динамической памятью намного более безопасной. Интеллектуальный указатель автоматически освобождает память, как только удаляется последний указатель на нее. В современных программах C++ следует использовать именно интеллектуальные указатели.

Термины

Деструктор (destructor). Специальная функция-член, которая освобождает занятую объектом память, когда он выходит из области видимости или удаляется.

Динамическая память (free store). Пул памяти, доступный программе для хранения объектов, создаваемых динамически.

Динамически созданный объект (dynamically allocated object). Объект, который создан в динамической памяти. Такие объекты существуют до тех пор, пока не будут удалены из динамической памяти явно или пока программа не завершит работу.

Интеллектуальный указатель `shared_ptr`. Интеллектуальный указатель, обеспечивающий совместную собственность: объект освобождается, когда удаляется последний указатель `shared_ptr` на тот объект.

Интеллектуальный указатель `unique_ptr`. Интеллектуальный указатель, обеспечивающий единоличную собственность: объект освобождается, когда удаляется указатель `unique_ptr` на этот объект. Указатель `unique_ptr` не может быть скопирован или присвоен непосредственно.

Интеллектуальный указатель `weak_ptr`. Интеллектуальный указатель на объект, управляемый указателем `shared_ptr`. Указатель `shared_ptr` не учитывает указатель `weak_ptr`, принимая решение об освобождении своего объекта.

Интеллектуальный указатель (smart pointer). Библиотечный тип, действует как указатель, но допускающий проверку на безопасность использования. Тип сам заботится об освобождении памяти, когда это нужно.

Класс allocator. Библиотечный класс, резервирующий области памяти.

Оператор delete. Освобождает участок памяти, зарезервированный оператором new. Оператор delete *p* освобождает объект, а delete [*i*] *p* — массив, на который указывает указатель *p*. Указатель *p* может быть пуст или указывать на область памяти, зарезервированную оператором new.

Оператор new. Резервирует область в динамической памяти. Оператор new *T* резервирует область памяти, создает в ней объект типа *T* и возвращает указатель на этот объект. Если *T* — тип массива, оператор new возвращает указатель на его первый элемент. Аналогично оператор new [*n*] *T* резервирует *n* объектов типа *T* и возвращает указатель на первый элемент массива. По умолчанию вновь созданный объект инициализируется значением по умолчанию. Но можно также предоставить инициализаторы.

Потерянный указатель (dangling pointer). Указатель, содержащий адрес области памяти, в которой уже нет объекта. Потерянный указатель является весьма распространенным источником ошибок в программе, причем такие ошибки крайне трудно обнаружить.

Размещающий оператор new (placement new). Форма оператора new, получающая в круглых скобках дополнительные аргументы после ключевого слова new; например, синтаксис new (nothrow) int устанавливает, что оператор new не должен передавать исключения.

Распределяемая память (heap). То же, что и динамическая память.

Счетчик ссылок (reference count). Счетчик, отслеживающий количество пользователей, совместно использующих общий объект. Используется интеллектуальными указателями, чтобы узнать, когда безопасно освободить память, на которую они указывают.

Функция удаления (deleter). Функция, передаваемая интеллектуальному указателю, для использования вместо оператора delete при освобождении объекта, на который указывает указатель.